

howest
hogeschool

WEB API

Programming Integration

INHOUD

1	INLEIDING	5
1.1	Wat is een API?	5
1.2	Verschil tussen web API en MVC	5
1.3	Voordelen van een API	6
1.4	API's voor developers	6
1.5	Communiceren met een API	6
2	REST AKA REPRESENTATIONAL STATE TRANSFER	7
2.1	Wat is REST?	7
2.2	Wanneer is een web service Restfull?	7
2.2.1	Uniforme interface	7
2.2.2	Stateless	7
2.2.3	Client-server separation	7
2.2.4	Layered system	7
2.3	Hoe communiceren met een REST Web API	8
2.3.1	HTTP methods/verbs	8
2.3.2	Endpoints	8
2.3.3	Afspraken omtrent endpoints	9
2.3.4	HTTP Response codes	9
2.3.4.1	2xx Succes	10
2.3.4.2	4xx Client Error	10
2.3.4.3	5xx Server Error	10
3	API PROJECT MAKEN	11
4	ENTITIES AANMAKEN	13
4.1	BaseEntity	13
4.2	Category	13
4.3	Product	13
5	ENTITY FRAMEWORK CORE CONFIGUREREN	14
5.1	NuGet packages toevoegen	14
5.2	Appsettings.json	14
5.3	ApplicationDbContext aanmaken	14
5.4	Program aanpassen voor DI van ApplicationDbContext	15
5.5	Migration toevoegen	15
5.6	Database updaten	16
6	SERVICE LAYER KLAARZETTEN	17
6.1	Result object	17
6.2	Interfaces/contracten definiëren	17
6.2.1	IProductService	17
6.2.2	ICategoryService	18
7	SERVICE IMPLEMENTATIES AANMAKEN	19
7.1	ProductService	19
7.2	CategoryService	22
8	DI CONFIGURATIE VOOR SERVICE KLASSEN	25

9	DATA TRANSFER OBJECTS (DTO'S)	26
9.1	Wat zijn DTO's?	26
9.2	Voorbeeld Book entity en BookDto	26
9.2.1	Book entity	26
9.2.2	Book DTO	27
9.3	Mapping Entity naar Dto (en vice versa)	27
9.4	Response en request DTO's	28
9.4.1	Response DTO	28
9.4.2	Request DTO	28
9.4.3	Voor- en nadelen	28
10	DTO'S TOEVOEGEN AAN HET API PROJECT	29
10.1	BaseDto	29
10.2	CategoryResponseDto	29
10.3	ProductResponseDto	29
11	API CONTROLLERS TOEVOEGEN	30
11.1	CategoriesController	30
11.1.1	Service injecteren	31
11.2	ProductsController	32
11.3	SearchController	32
12	HTTP GET REQUESTS	33
12.1	Collectie van categorieën opvragen	33
12.1.1	Endpoint uittesten	34
12.2	Categorie opvragen op basis van ID	34
12.2.1	Endpoint uittesten	35
12.3	Collectie van producten opvragen	36
12.3.1	Endpoint uittesten	37
12.4	Product opvragen op basis van ID	38
12.4.1	Endpoint uittesten	39
12.5	Alle Producten van één categorie opvragen op basis van ID	39
	Endpoint uittesten	40
12.6	Zoeken op categorie en/of product	40
12.6.1	Endpoint uittesten	41
13	HTTP POST REQUESTS	42
13.1	CategoryRequestDto aanmaken	42
13.2	ProductRequestDto aanmaken	42
13.3	Categorie toevoegen	43
13.3.1	Endpoint uittesten met Postman	44
13.4	Product toevoegen	47
13.4.1	Endpoint uittesten met Postman	49
14	HTTP PUT REQUESTS	52
14.1	Categorie aanpassen	52
14.1.1	Endpoint uittesten met Postman	53
14.2	Product aanpassen	56
14.2.1	Endpoint uittesten met Postman	57

15	HTTP DELETE REQUESTS	58
15.1	Categorie verwijderen	58
15.1.1	Endpoint uittesten met Postman	59
15.2	Product verwijderen	60
	Endpoint uittesten met Postman	60
16	EEN FILE UPLOADEN	61
	uittesten met swagger	65
17	CORS (CROSS ORIGIN)	67

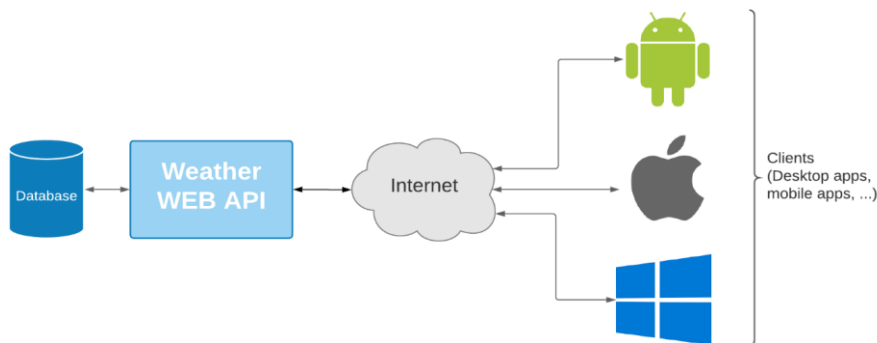
1 INLEIDING

Web application programming interfaces, of kortweg web API, zijn hedendaags een essentiële pijler in onze geconnecteerde wereld.

Verskillende soorten software gebruiken deze application programming interfaces om te communiceren met backend servers. Denk hierbij aan je een app op je smartphone, je tablet, je pc, je slimme thermostaat, ... die allemaal communiceren met een web API. Web API's zijn hedendaags overal!

1.1 WAT IS EEN API?

Elk besturingssysteem, zowel op desktop, mobile, tablet, ... bevat wel een weer applicatie om de weersvoorzichten te bekijken. Deze weerapplicaties op deze toestellen berekenen de vooruitzichten NIET op de applicaties (ook wel de clients genoemd als we over API spreken) maar dit gebeurt op een backend server die we de web API noemen. De clients spreken dus via het HTTP-protocol de web API aan over het internet. Deze server communiceert op zijn beurt met een database om data op te halen of te manipuleren. De clients communiceren via het internet met deze server en ontvangen data van deze server over het internet.



De ontvangen data is pure data in de vorm van JSON, XML, ... De ontvangen data wordt in een mooie user interface gegoten op de client zelf zodat de gebruiker een mooie grafische ervaring geniet op deze client.



Clients

Als we tijdens dit hoofdstuk spreken over clients, hou er dan rekening mee dat dit een algemeen begrip is. Clients kunnen mobile apps, desktop apps, browsers, Internet Of Things-apparaten, ... zijn. In bovenstaand schema staan logo's van Android, Apple en Windows ter illustratie maar clients zijn zeker en vast niet beperkt tot deze besturingssystemen en/of apparaten!

1.2 VERSCHIL TUSSEN WEB API EN MVC

Programmeer technisch is er weinig verschil tussen een MVC en een web API applicatie. Wanneer we naar een URL surfen bij een MVC applicatie krijgen we een mooie View te zien met de nodige HTML, CSS en Javascript om bepaalde data weer te geven met een mooie grafische layout.

Bij het opvragen van gegevens bij een web API krijgen we enkel en alleen data teruggestuurd. Geen HTML maar JSON, XML of een ander data formaat.

Een kleine opsomming met verschillen tussen MVC en API waar later nog op terug zal gekomen worden:

- Een web API bevat controllers en models maar geen views.
- Models in een API noemen geen ViewModels maar Data Transfer Objects (DTO's)
- Een MVC applicatie verstuurt HTML als response naar de clients (meestal browsers)
- Een API verstuurt DATA naar de clients

Een MVC applicatie kan ook een client zijn die kan communiceren met een API en de ontvangen data voorzien van HTML en CSS voor een mooie opmaak.

1.3 VOORDELEN VAN EEN API

Wanneer we gebruik maken van een API hoeven we alle business logic, aanspreken van een database, ... slechts te voorzien in de API en niet per client.

1.4 API'S VOOR DEVELOPERS

Bijna alle grote tech bedrijven bieden een API aan zodat developers hiermee aan de slag kunnen gaan voor eigen applicaties. Echter moet je je meestal registreren (en soms betalen) bij deze bedrijven als developer om toegang te krijgen tot hun API's. De reden hiervoor dat deze bedrijven toch wat controle kunnen houden op het aanspreken van hun API's zodat er geen misbruik van gemaakt wordt.

- Twitter (<https://developer.twitter.com/en/docs/twitter-api/getting-started/guide>)
- Spotify (<https://developer.spotify.com/documentation/web-api/>)
- Discord (<https://discord.com/developers/docs/intro>)
- GitHub (<https://docs.github.com/en/rest>)
- En nog vele andere vind je hier: <https://apilist.fun>

1.5 COMMUNICEREN MET EEN API

Aangezien bijna alle applicaties gebruik maken van een web API is er een set van protocollen, definities en afspraken gemaakt om applicaties te laten communiceren met API's. Deze methode van communiceren heet REST en staat voor Representational State Transfer. We bespreken dit in het volgende hoofdstuk.

2 REST AKA REPRESENTATIONAL STATE TRANSFER

De resource-centric benadering en de eenvoud van de REST-stijl zijn een industriestandaard geworden. Daarom is het essentieel om de theorie achter de REST-manier van werken te begrijpen en waarom deze belangrijk is. We zullen zien wat de definitie van REST is en hoe REST-compatibele webservices kunnen worden geïdentificeerd.

2.1 WAT IS REST?

REST staat voor **Representational State Transfer**.

De gedachte achter REST is dat we onze services zien als bronnen van data, die via een eenduidig adres en een statusloos (stateless) protocol (http) te raadplegen zijn.

Met REST is het vooral de bedoeling algemene afspraken te maken hoe we met web services omgaan, het is geen nieuwe technologie op zich.

De REST-benadering van webservice-architectuur werd steeds populairder omdat deze eenvoudig en duidelijk is.

2.2 WANNEER IS EEN WEB SERVICE RESTFULL?

Om een RESTfull web service, in ons geval dus een API, te zijn moet voldaan worden aan enkele verplichtingen.

2.2.1 UNIFORME INTERFACE

Uniforme interface verwijst naar de scheiding tussen de client en de server. Dit is voordelig omdat het betekent dat de twee systemen onafhankelijk zijn. Het eerste principe van een uniforme interface is dat deze resource-based moet zijn, wat betekent dat we resource-georiënteerd moeten gaan denken. Daarom is elk object of elke entiteit in ons systeem een bron en een URI identificeert elk van hen op unieke wijze. Bovendien, als we nadenken over het HTTP-protocol, wordt elke bron aan de client gepresenteerd in de vorm van XML of JSON om de client los te koppelen van de server.

2.2.2 STATELESS

Stateless zijn is een cruciaal kenmerk van een REST-service. HTTP, als een stateless protocol, houdt niet alle informatie bij over de verbinding tussen de client en de server zodra de communicatie is beëindigd.

Een stateless protocol dwingt de client om het verzoek te vervullen met alle vereiste informatie elke keer dat de client data van de server nodig heeft.

2.2.3 CLIENT-SERVER SEPARATION

Het belangrijkste doel van REST-services is om de server en de client te ontkoppelen. Dit is erg belangrijk omdat het helpt om unieke business logic en gegevensopslag voor elke clienttoepassing te behouden. Applicaties worden meestal gedistribueerd over een groot aantal verschillende klanten, waaronder internet, smartphones, smart-tv's en het IoT.

Een REST-benadering helpt ons logische replicatie tussen clients te voorkomen. Dit betekent dat de client geen business logic of opslag heeft en dat de server niet te maken heeft met gebruikersinterfaces of de presentatielaag.

2.2.4 LAYERED SYSTEM

Het concept van een gelaagd systeem is strikt gerelateerd aan de structuur van de infrastructuur van onze applicatie. RESTful-services maken een loosely coupled approach mogelijk omdat de informatie wordt

overgedragen via een protocol - in de meeste gevallen HTTP - en elke server een enkel doel op hoog niveau heeft.

Proxyservers, webserver en databaseservers zijn meestal geïsoleerd en hebben een doel in onze functie: als u één server hebt die alle vereiste functies biedt, is deze vaak moeilijk te onderhouden en te schalen.

2.3 HOE COMMUNICEREN MET EEN REST WEB API

Aangezien we gebruik maken van het HTTP protocol gaan we dus de verschillende HTTP-methoden (of ook wel HTTP-verbs genoemd) gebruiken.

2.3.1 HTTP METHODS/VERBS

HttpMethod	Actie	Omschrijving
GET	R(ead) R(etrieve)	Data opvragen van de databron
POST	C(reate)	Data toevoegen
PUT	U(pdate)	Data editen / toevoegen / andere verwerkingen
DELETE	D(elete)	Data verwijderen

2.3.2 ENDPOINTS

Aangezien één van de hoofdgedachten van REST is om services aan te bieden via een uniforme en eenduidige manier gaan we dit doen aan de hand van een gestructureerde URL. Een API URL wordt meestal beschreven als een API endpoint. Wat dus eigenlijk een gewone url is volgens een bepaalde structuur.

Een API endpoint kan je meestal herkennen aan het API woord in de URL, bijvoorbeeld <https://www.mijnshop.be/api/> of <https://api.mijnshop.be/>

In deze cursus maken we gebruik van <https://mijn-url.be/api/> waarin api achteraan onze root url staat.

Enpoint	methode	Beschrijving	Request	Response
<code>/api/Products</code>	GET	Geef een lijst van producten	Niets	Collectie van producten
<code>/api/Products/2</code>	GET	Geef de details van product met id 2	Niets	Product met ID 2
<code>/api/Products</code>	PUT	Bewerk de gegevens van een product	Product	Niets
<code>/api/Products</code>	POST	Voeg een product toe	Product	Product
<code>/api/Products/2</code>	DEL	Verwijder een product	Niets	Niets

We zijn dus dat het endpoint `/api/Products` vier keer voorkomt, het zijn echter de HTTP-verbs (GET, PUT, POST en DELETE) die voor de nuance zorgen.

2.3.3 AFSPRAKEN OMTRENT ENDPOINTS

- We voorzien alles in het meervoud! Dus `products` en geen `product`.
- De endpoint bevat de resource/data/entity naam (in het meervoud)
- Heb je alle producten van een bepaalde categorie nodig dan zal je endpoint er als volgt uitzien: <https://www.mijnshop.be/api/categories/{id-van-category}/products> met een HTTP GET.



Endpoints

Bovenstaande REST afspraken zijn duidelijk genoeg. **Maak dus geen endpoints zoals:**

- `/api/DeleteProduct/` met een HTTP Delete
- `/api/UpdateProduct/` met een HTTP Put
- `/api/AddNewProduct/` met een HTTP Post
- `/api/GetAllProductsFromCategory/` met een HTTP Get

Indien je dit wel zou doen zondig je tegen de REST principes!!

Bij het naleven van deze afspraken weet iedereen die je API wenst te gebruiken hoe deze aangesproken moet worden, ongeacht welke CRUD actie de gebruiker wenst te doen.

Op die manier kunnen we globale afspraken maken om bvb. CRUD (Create, Retrieve, Update, Delete)-toepassingen te raadplegen.

2.3.4 HTTP RESPONSE CODES

Voor de gebruiker van je API is het erg belangrijk dat je de correcte HTTP response codes terugstuurt zodat de gebruiker weet of een request al of niet gelukt is. En vooral; indien het niet gelukt is de reden waarom het niet gelukt is.

HTTP response codes kunnen in vijf groepen ingedeeld worden:

1. 1xx Informational
2. 2xx Success
3. 3xx Redirection
4. 4xx Client Error
5. 5xx Server Error

We bespreken hieronder enkele veelvoorkomende codes, maar hou er rekening mee dat er nog meer zijn dan diegene die we hier bespreken. Zie hier een volledige lijst:

<https://www.restapitutorial.com/httpstatuscodes.html>

2.3.4.1 2XX SUCCES

Deze statuscodes geven aan dat iets succesvol is verlopen.

200 OK

De request is succesvol.

201 CREATED

De request is succesvol verlopen en er werd een nieuwe resource aangemaakt. (Bij een POST request)

2.3.4.2 4XX CLIENT ERROR

400 BAD REQUEST

De request werd niet begrepen door de server door verkeerdelijke syntax.

401 UNAUTHORIZED

De request vereist een geautoriseerde gebruiker en is dit nu niet.

403 FORBIDDEN

De request is succesvol maar de gebruiker heeft geen toegang tot de endpoint door zijn autorisatie niveau.

404 NOT FOUND

De server heeft geen passende route gevonden voor de gevraagde request OF er werd geen data gevonden voor de gevraagde request.

2.3.4.3 5XX SERVER ERROR

500 INTERNAL SERVER ERROR

De server kwam in een ongewenste toestand terecht en kan de request niet vervullen. (Dikwijls is de server gecrashed of gooit deze een exception op)

3 API PROJECT MAKEN

Open Visual Studio en maak een nieuw project aan.

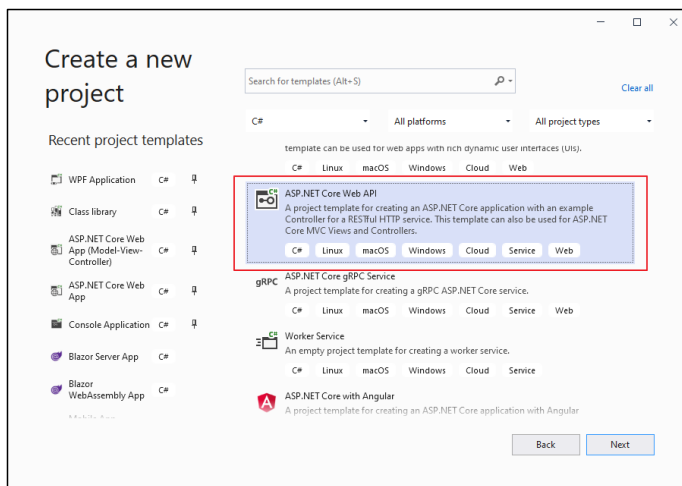


Code repository

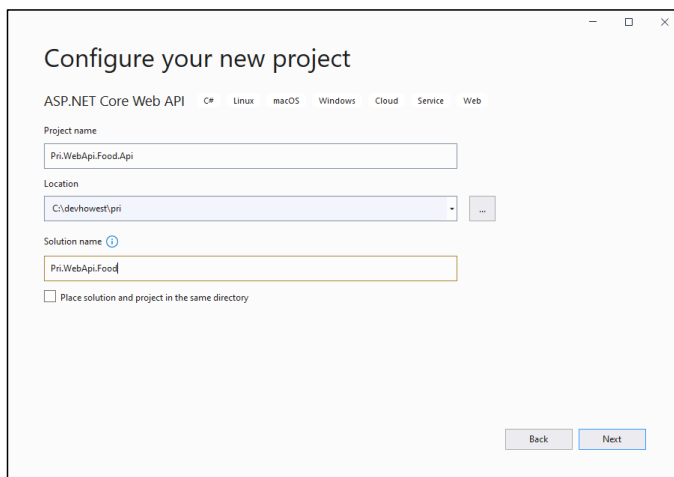
De volledige broncode van dit voorbeeld is te vinden op

`git clone` <https://classroom.github.com/a/ytcUCvDN>

Kies als template voor **ASP.NET Core Web API**:



Geef een gepaste naam aan je project en solution:



Voorzie de nodige instellingen:

Additional information

ASP.NET Core Web API C# Linux macOS Windows Cloud Service Web

Target Framework ⓘ

Huidige versie .NET

Authentication Type ⓘ

None

☒ Configure for HTTPS ⓘ

☐ Enable Docker ⓘ

Docker OS ⓘ

Linux

☒ Enable OpenAPI support ⓘ

Back

Create

4 ENTITIES AANMAKEN

- Maak in je solution een nieuwe Class Library aan en noem het Pri.WebApi.Food.Core.
- Voeg in het Api-project een referentie toe naar de .Core-bibliotheek.
(Rechtermuisknop op je solution > Add > New Project ... > Class Library (in de huidige versie .NET))
- Voeg in het .Core project een nieuwe folder **Entities** toe en maak hierin drie klassen aan:

4.1 BASEENTITY

Maak een abstracte klasse **BaseEntity** aan met drie properties en voorzie de nodige using statement.

```
public abstract class BaseEntity
{
    public Guid Id { get; set; }

    [Required]
    public DateTime CreatedOn { get; set; }

    [Required]
    public DateTime LastEditedOn { get; set; }
}
```

4.2 CATEGORY

Maak een klasse **Category** aan die overerft van **BaseEntity** en voorzie een **Name** property. Voorzie de nodige using statements.

```
public class Category : BaseEntity
{
    [Required]
    public string Name { get; set; }
}
```

4.3 PRODUCT

Maak een klasse **Product** aan die overerft van **BaseEntity**. Voorzie een **Name** property, een **CategoryId** property en een **Category** property. Voorzie de nodige using statements.

```
public class Product : BaseEntity
{
    [Required]
    public string Name { get; set; }

    [Required]
    public Guid CategoryId { get; set; }

    public Category Category { get; set; }
    public string Image { get; set; }
}
```

5 ENTITY FRAMEWORK CORE CONFIGUREREN

5.1 NUGET PACKAGES TOEVOEGEN

Voorzie in het API project volgende NuGet packages toe:

- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.Design
- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Relational

5.2 APPSETTINGS.JSON

Voeg de database connectie toe in je **appsettings.json** bestand:

```
{
  "ConnectionStrings": {
    "DefaultDatabase": "Server=(localdb)\\mssqllocaldb;Database=Pri.WebApi.Food;Trusted_Connection=True;MultipleActiveResultSets=true"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*"
}
```

5.3 APPLICATIONDBCONTEXT AANMAKEN

Maak in je het .Core project een nieuwe folder aan met de naam **Data** en maak hierin een nieuwe klasse met de naam **ApplicationDbContext**.

- Laat deze overerven van DbContext
- Voorzie de nodige constructor
- Voorzie twee DbSet properties voor Products en Categories
- Voorzie een override OnModelCreating methode met de nodige seeding data
- Voorzie de nodige using statements

```

public class ApplicationDbContext : DbContext
{
    public DbSet<Product> Products { get; set; }
    public DbSet<Category> Categories { get; set; }

    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
base(options)
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Category>().HasData(
            // Data aan te vullen via onderstaande link
        );

        modelBuilder.Entity<Product>().HasData(
            // Data aan te vullen via onderstaande link
        );
    }
}

```



Code repository

De nodige seeding data is hier te vinden:

 <https://github.com/howest-gp-pri/cu-webapi-food/blob/master/Pri.WebApi.Food.Api/Data/ApplicationDbContext.cs>

Commented [ML1]: Updaten wannen nieuwe voorbeeld online?

5.4 PROGRAM AANPASSEN VOOR DI VAN APPLICATIONDBCONTEXT

Voeg de **AddDbContext** methode toe om je **ApplicationDbContext** te configureren voor DI.

```

builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultDatabase"))
);

```

5.5 MIGRATION TOEVOEGEN

Build je solution nogmaals en voeg nu een migration toe:

```
dotnet ef migrations add "InitialMigration"
```

```
PowerShell 7
C:\devhowest\pri\Pri.WebApi.Food\Pri.WebApi.Food.Api
> dotnet ef migrations add "InitialMigration"
Build started...
Build succeeded.
The Entity Framework tools version '5.0.3' is older than that of the runtime '5.0.10'. Update the tools for the latest features and bug fixes.
Done. To undo this action, use 'ef migrations remove'

C:\devhowest\pri\Pri.WebApi.Food\Pri.WebApi.Food.Api
> _
```

5.6 DATABASE UPDATEN

Update nu je database:

```
dotnet ef database update
```

```
PowerShell 7
C:\devhowest\pri\Pri.WebApi.Food\Pri.WebApi.Food.Api
> dotnet ef database update
Build started...
Build succeeded.
The Entity Framework tools version '5.0.3' is older than that of the runtime '5.0.10'. Update the tools for the latest features and bug fixes.
Done.

C:\devhowest\pri\Pri.WebApi.Food\Pri.WebApi.Food.Api
> _
```


6 SERVICE LAYER KLAARZETTEN

- Maak in je het .Core-project een nieuwe folder aan met de naam **Services**.
- Voorzie in deze folder een nieuwe folder **Interfaces**
- Voorzie bovendien een folder **Models**

6.1 RESULT OBJECT

Maak nu in de **Models** folder een klasse genaamd **BaseResult**. Dit is een klasse die verantwoordelijk gesteld wordt om aan te geven of een operatie in onze applicatie al dan niet gelukt is (Succes). In geval van errors, dan houden we die ook bij zodat we deze straks kunnen terugsturen naar de clients om hen op de hoogte te stellen van eventuele fouten:

```
public abstract class BaseResult
{
    public bool Success => !Errors.Any();
    public List<string> Errors { get; set; } = new List<string>();
}
```

Vervolgens gebruiken we deze BaseResult class om een ResultModel class te implementeren door middel van generics om een Data property dynamisch te kunnen instellen op de gewenste entity class.

```
public class ResultModel<T> : BaseResult
{
    public T Data { get; set; }
}
```

6.2 INTERFACES/CONTRACTEN DEFINIËREN

6.2.1 IPRODUCTSERVICE

Maak in de folder **Interfaces** een interface aan met de naam **IProductService**. Voorzie volgende code:

```
public interface IProductService
{
    IQueryable<Product> GetAll();
    Task<ResultModel<IEnumerable<Product>>> ListAllAsync();
    Task<ResultModel<Product>> GetByIdAsync(Guid id);
    Task<ResultModel<Product>> UpdateAsync(Product entity);
    Task<ResultModel<Product>> AddAsync(Product entity);
    Task<ResultModel<Product>> DeleteAsync(Product entity);
    Task<ResultModel<IEnumerable<Product>>> GetByCategoryIdAsync(Guid id);
    Task<ResultModel<IEnumerable<Product>>> SearchAsync(string search);
    Task<bool> DoesProductIdExistAsync(Guid id);
    Task<bool> DoesProductNameExistsAsync(Product entity);
}
```

6.2.2 ICATEGORYSERVICE

Maak in de folder Interfaces een tweede interface aan met de naam **ICategoryService** en voorzie de volgende code:

```
public interface ICategoryService
{
    IQueryable<Category> GetAll();
    Task<ResultModel<IEnumerable<Category>>> ListAllAsync();
    Task<ResultModel<Category>> GetByIdAsync(Guid id);
    Task<bool> DoesCategoryIdExistsAsync(Guid id);
    Task<bool> DoesCategoryNameExistsAsync(Category entity);
    Task<ResultModel<IEnumerable<Category>>> GetByName(string name);
    Task<ResultModel<Category>> UpdateAsync(Category entity);
    Task<ResultModel<Category>> AddAsync(Category entity);
    Task<ResultModel<Category>> DeleteAsync(Category entity);
}
```

7 SERVICE IMPLEMENTATIES AANMAKEN

Maak in de bestaande folder **Services** twee nieuwe klassen, **CategoryService** en **ProductService**, aan. Voorzie in beiden een injectie van de **ApplicationDbContext**. Deze services zullen dus verantwoordelijk gesteld worden om logica of operaties te bevatten waarvoor database-bevragingen nodig zijn.

7.1 PRODUCTSERVICE

Voorzie in **ProductService** de volgende code:

```
public class ProductService : IProductService
{
    private readonly ApplicationDbContext _applicationDbContext;
    public ProductService(ApplicationDbContext applicationDbContext)
    {
        _applicationDbContext = applicationDbContext;
    }

    public async Task<ResultModel<Product>> AddAsync(Product entity)
    {
        //does product with same name exist?
        if(await DoesProductNameExistsAsync(entity))
        {
            new ResultModel<Product>
            {
                Errors = new List<string>
                { $"There is already a product with name = {entity.Name}" }
            };
        }
        entity.CreatedOn = DateTime.UtcNow;
        entity.LastEditedOn = DateTime.UtcNow;

        _applicationDbContext.Products.Add(entity);
        await _applicationDbContext.SaveChangesAsync();

        return new ResultModel<Product> { Data = entity };
    }

    public async Task<ResultModel<Product>> DeleteAsync(Product entity)
    {
        _applicationDbContext.Products.Remove(entity);
        await _applicationDbContext.SaveChangesAsync();
        return new ResultModel<Product>
        {
            Data = entity
        };
    }

    public async Task<bool> DoesProductIdExistAsync(Guid id)
    {
        return await _applicationDbContext.Products.AnyAsync(p => p.Id.Equals(id));
    }
}
```

```

public async Task<bool> DoesProductNameExistsAsync(Product entity)
{
    return await _applicationDbContext.Products
        .Where(p => p.Id != entity.Id)
        .AnyAsync(p => p.Name.Equals(entity.Name));
}

public IQueryable<Product> GetAll()
{
    return _applicationDbContext.Products.Include(p => p.Category);
}

public async Task<ResultModel<IEnumerable<Product>>> GetByCategoryIdAsync(Guid id)
{
    var products = await _applicationDbContext.Products
        .Include(p => p.Category)
        .Where(p => p.CategoryId.Equals(id)).ToListAsync();
    return new ResultModel<IEnumerable<Product>> { Data = products };
}

public async Task<ResultModel<Product>> GetByIdAsync(Guid id)
{
    var product = await _applicationDbContext
        .Products
        .Include(p => p.Category)
        .FirstOrDefaultAsync(p => p.Id == id);
    if(product is null)
    {
        return new ResultModel<Product>
        {
            Errors = new List<string> { "Product does not exist" }
        };
    }
    return new ResultModel<Product> { Data = product };
}

public async Task<ResultModel<IEnumerable<Product>>> ListAllAsync()
{
    var products = await _applicationDbContext
        .Products
        .Include(p => p.Category)
        .ToListAsync();
    var resultModel = new ResultModel<IEnumerable<Product>>()
    {
        Data = products,
    };
    return resultModel;
}

public async Task<ResultModel<IEnumerable<Product>>> SearchAsync(string search)
{
    var products = await _applicationDbContext.Products
        .Include(p => p.Category)
        .Where(p => p.Name
            .Contains(search.Trim())
            || p.Category.Name.Contains(search.Trim()))
        .ToListAsync();
    return new ResultModel<IEnumerable<Product>> { Data = products };
}

```

```

public async Task<ResultModel<Product>> UpdateAsync(Product entity)
{
    //product id exists?
    if(!await DoesProductIdExistAsync(entity.Id))
    {
        return new ResultModel<Product>
        {
            Errors = new List<string> { $"There is no product with id {entity.Id}!" }
        };
    }
    //does product name exist
    if (await DoesProductNameExistsAsync(entity))
    {
        return new ResultModel<Product>
        {
            Errors = new List<string>
            { $"There is already a product with name {entity.Name}!" }
        };
    }
    //update the entity
    entity.LastEditedOn = DateTime.UtcNow;
    _applicationDbContext.Products.Update(entity);
    await _applicationDbContext.SaveChangesAsync();

    return new ResultModel<Product>
    {
        Data = entity
    };
}
}

```

Deze **ProductService** is de meest eenvoudige. Doordat de service de **ApplicationDbContext** injecteert kunnen we alle database-operaties (hier: alle CRUD-operaties) erin onderbrengen, maar toch bevat ze al wat code die complexere bevestigingen doet:

- **SearchAsync**: filtert op basis van een zoekterm de resultaten
- **GetByCategoryIdAsync**: betreft de bijhorende Category om alle producten binnen een bepaalde category te bekomen
- **DoesProductExistAsync**: Controleert op basis van een product id of het product bestaat
- **DoesProductNameExistAsync**: Controleert of een productnaam reeds bestaat inde database

7.2 CATEGORYSERVICE

Voorzie in de **CategoryService** de volgende code. Merk op dat sommige methodes ditmaal puur dienen als checks. Deze retourneren dan ook een **Result** object met info of de check een positief of negatief resultaat oplevert. In geval van een negatieve check, wordt een lijst van error(s) teruggestuurd.

```
public class CategoryService : ICategoryService
{
    private readonly ApplicationDbContext _applicationDbContext;
    public CategoryService(ApplicationDbContext applicationDbContext)
    {
        _applicationDbContext = applicationDbContext;
    }

    public async Task<ResultModel<Category>> AddAsync(Category entity)
    {
        //does categoryname exist?
        if(await DoesCategoryNameExistsAsync(entity))
        {
            return new ResultModel<Category>
            {
                Errors = new List<string>
                { $"A category with the name {entity.Name} already exists!" },
            };
        }
        entity.CreatedOn = DateTime.UtcNow;
        entity.LastEditedOn = DateTime.UtcNow;
        _applicationDbContext.Categories.Add(entity);
        await _applicationDbContext.SaveChangesAsync();
        return new ResultModel<Category> { Data = entity };
    }

    public async Task<ResultModel<Category>> DeleteAsync(Category entity)
    {
        _applicationDbContext.Categories.Remove(entity);
        await _applicationDbContext.SaveChangesAsync();
        return new ResultModel<Category> { Data = entity };
    }

    public async Task<bool> DoesCategoryIdExistsAsync(Guid id)
    {
        return await _applicationDbContext.Categories.AnyAsync(c => c.Id.Equals(id));
    }

    public async Task<bool> DoesCategoryNameExistsAsync(Category Entity)
    {
        return await _applicationDbContext.Categories
            .Where(c => c.Id != Entity.Id)
            .AnyAsync(c => c.Name.Equals(Entity.Name));
    }

    public IQueryable<Category> GetAll()
    {
        return _applicationDbContext.Categories;
    }
}
```

```

public async Task<ResultModel<Category>> GetByIdAsync(Guid id)
{
    var resultModel = new ResultModel<Category>();
    var category = await _applicationDbContext.Categories
        .FirstOrDefaultAsync(category => category.Id.Equals(id));
    if (category is null)
    {
        resultModel = new ResultModel<Category>();
        resultModel.Errors.Add($"Category does not exists");
        return resultModel;
    }
    resultModel = new ResultModel<Category> { Data = category };
    return resultModel;
}

public async Task<ResultModel<IEnumerable<Category>>> GetByName(string name)
{
    var categories = await _applicationDbContext.Categories
        .Where(c => c.Name.Contains(name)).ToListAsync();
    return new ResultModel<IEnumerable<Category>> { Data = categories };
}

public async Task<ResultModel<IEnumerable<Category>>> ListAllAsync()
{
    var categories = await _applicationDbContext.Categories.ToListAsync();
    return new ResultModel<IEnumerable<Category>> { Data = categories };
}

public async Task<ResultModel<Category>> UpdateAsync(Category entity)
{
    //does category id exist?
    if(!await DoesCategoryIdExistsAsync(entity.Id))
    {
        return new ResultModel<Category>
        {
            Errors = new List<string> { $"There is no category with id :{entity.Id}" }
        };
    }
    //does another category with same name exist?
    if(await DoesCategoryNameExistsAsync(entity))
    {
        return new ResultModel<Category>
        {
            Errors = new List<string> { $"There is already category with id :{entity.Id}" }
        };
    }
    entity.LastEditedOn = DateTime.UtcNow;
    _applicationDbContext.Categories.Update(entity);
    await _applicationDbContext.SaveChangesAsync();
    return new ResultModel<Category>
    {
        Data = entity
    };
}
}

```

Ook hier zijn een aantal methodes aanwezig die een aantal controles uitvoeren:

- **DoesCategoryIdExistAsync**: controleert of een Category bestaat aan de hand van het id
- **DoesCategoryNameExist**: controleert of een category name reeds bestaat.

8 DI CONFIGURATIE VOOR SERVICE KLASSEN

Voorzie in de **Program** klasse nu volgende code om de DI te configureren

```
builder.Services.AddScoped<IProductService, ProductService>();  
builder.Services.AddScoped<ICategoryService, CategoryService>();
```

9 DATA TRANSFER OBJECTS (DTO'S)

Aangezien een Web API geen Views bevat, zal een Web API dus ook geen ViewModels bevatten. Toch dienen we onze data die we naar de clients wensen te versturen (of data die we ontvangen van de clients) in een object te verzamelen.

9.1 WAT ZIJN DTO'S?

Bij een API gaan we hiervoor zogenaamde Data Transfer Objects (DTO's) aanmaken. Het zijn namelijk deze objecten die verstuurt zullen worden naar onze clients.

Zoals je reeds weet uit Web Backend gaan we nooit de database entities zelf gaan versturen naar de clients. De clients hoeven niet alle data uit de database te ontvangen.

We gaan dus de data die we naar een client zullen sturen wat gaan aanpassen zodat wij als developer controle hebben over welke data we precies (willen) terugsturen en in welke vorm we dat (willen) doen.

Enkele redenen hiervoor kunnen zijn:

- Verwijderen van circular references van objecten
- Verbergen van bepaalde properties van entities die clients niet hoeven of mogen zien
- "Platter" (flatten) maken van objecten die geneste objecten bevatten
- Bovendien kunnen data uit één of meerdere entities gaan toevoegen aan een DTO. Denk hierbij aan een eigenschap 'NumberOfTracks' in een AlbumDto. Deze 'NumberOfTracks' kan afkomstig zijn uit een collectie van Tracks, maar wordt in de DTO weergegeven als de som van het aantal tracks, niet als de eigenlijke collectie.

Heel geregeld zal het echter gebeuren dat een DTO (bijna) een één op één kopie is van een entity. Ook dan maken we een DTO aan.

9.2 VOORBEELD BOOK ENTITY EN BOOKDTO

Om het verschil tussen een entity en DTO uit te leggen gaan we even twee fictieve klassen bekijken.

9.2.1 BOOK ENTITY

Een Book entity kan volgende properties en dus ook volgende data bevatten in een database:

```
public class Book
{
    public int Id { get; set; }
    [Required]
    public string Title { get; set; }
    public int Year { get; set; }
    public decimal Price { get; set; }
    public string Genre { get; set; }

    // Foreign Key
    public int AuthorId { get; set; }
    // Navigation property
    public Author Author { get; set; }
}
```

9.2.2 BOOK DTO

Maar aan de clients die boeken opvragen wensen we enkel maar volgende data te tonen:

- Id
- Title
- AuthorName

Dus hiervoor maken we een DTO object aan:

```
public class BookDto
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string AuthorName { get; set; }
}
```

We wensen dus de properties Year, Price, Genre, ArtistId en Artist properties niet te tonen aan onze clients. Enkel van de navigation property Artist hebben we wel de Name property nodig om deze te kunnen tonen in de BookDto property AuthorName.

9.3 MAPPING ENTITY NAAR DTO (EN VICE VERSA)

We zullen dus Book moeten gaan “mappen” naar BookDto.

Stel dat we een lijst van alle boeken in de database wensen te versturen naar een client. Dan kunnen we de mapping van Book naar BookDto gaan uitvoeren doormiddel van de LINQ method **.Select()**.

Bekijk eventjes onderstaand voorbeeld:

```
// GET api/Books
public IQueryable<BookDto> GetBooks()
{
    var books = _dbContext.Books
        .Include(b => b.Author)
        .Select(b => new BookDto
        {
            Id = b.Id,
            Title = b.Title,
            AuthorName = b.Author.Name
        });

    return books;
}
```

Je kan je wel al voorstellen dat bij entities met véél properties en geneste objecten de Select method al redelijk snel groot en complex kan worden. Zeker als je ook de omgekeerde mapping wenst te voorzien van BookDto naar Book entity.

Wanneer je regelmatig dezelfde lambda expressies dient te schrijven in de Select method dan kan je beter een extension method schrijven waarin je de mapping voorziet. Of je gebruikt hiervoor een externe library zoals AutoMapper. Het gebruik van AutoMapper bespreken we niet in deze cursus.

9.4 RESPONSE EN REQUEST DTO'S

Bij een API zullen we onze DTO's per entity gaan opsplitsen in een response en request DTO. De reden hiervoor is dat een DTO die we ontvangen van een client niet altijd dezelfde properties dient te bevatten als een DTO die we versturen naar een client.

9.4.1 RESPONSE DTO

Een response DTO wordt verstuurd van de API naar de client

9.4.2 REQUEST DTO

Een request DTO wordt ontvangen door onze API en dus verstuurd door een client naar onze API.

9.4.3 VOOR- EN NADELEN

Deze opsplitsing heeft zowel voor- als nadelen.

Voordelen:

- Weinig properties per DTO
- Gemakkelijk aanpasbaar of uit te breiden

Nadelen:

- Meer DTO's in je project
- Meer code schrijven

Echter wegen op termijn de nadelen niet op tegen de voordelen, ook al heb je te maken met een erg klein project met weinig endpoints in je API.

10 DTO'S TOEVOEGEN AAN HET API PROJECT

Maak in het .Api-project een nieuwe folder met de naam **Dtos**.

10.1 BASEDTO

Maak een nieuwe **abstracte** klasse aan met de naam **BaseDto**. Voorzie een property voor het **Id**.

Merk op dat we de properties **CreatedOn** en **LastEditedOn** niet voorzien in deze DTO, ook al zijn die wel aanwezig in onze **BaseEntity**. Hier simuleren we dus een scenario waarin we deze properties wel wensen bij te houden in onze database, maar deze niet wensen te tonen aan onze clients die de data zullen opvragen.

Voorzie volgende code:

```
public abstract class BaseDto
{
    public Guid Id { get; set; }
}
```

10.2 CATEGORYRESPONSEDTO

Maak in de folder **Dtos** een nieuwe folder aan met de naam **Categories**.

Voorzie een nieuwe klasse met de naam **CategoryResponseDto**. Deze DTO zal de data bevatten die we naar onze clients gaan sturen die één categorie of een collectie van categorieën wensen op te vragen.

Laat deze DTO overerven van **BaseDto**.

Voorzie volgende code:

```
public class CategoryResponseDto : BaseDto
{
    public string Name { get; set; }
}
```

10.3 PRODUCTRESPONSEDTO

Maak in de folder **Dtos** een nieuwe folder aan met de naam **Products**.

Voorzie een nieuwe klasse met de naam **ProductResponseDto**. Deze DTO zal de data bevatten die we naar onze clients gaan sturen die één product of een collectie van producten wensen op te vragen. In de response zullen we ook de **Category** van dit product voorzien (van het type **CategoryResponseDto**)

Laat deze DTO overerven van **BaseDto**.

Voorzie volgende code:

```
public class ProductResponseDto : BaseDto
{
    public string Name { get; set; }
    public CategoryResponseDto Category { get; set; }
    public string Image { get; set; }
}
```

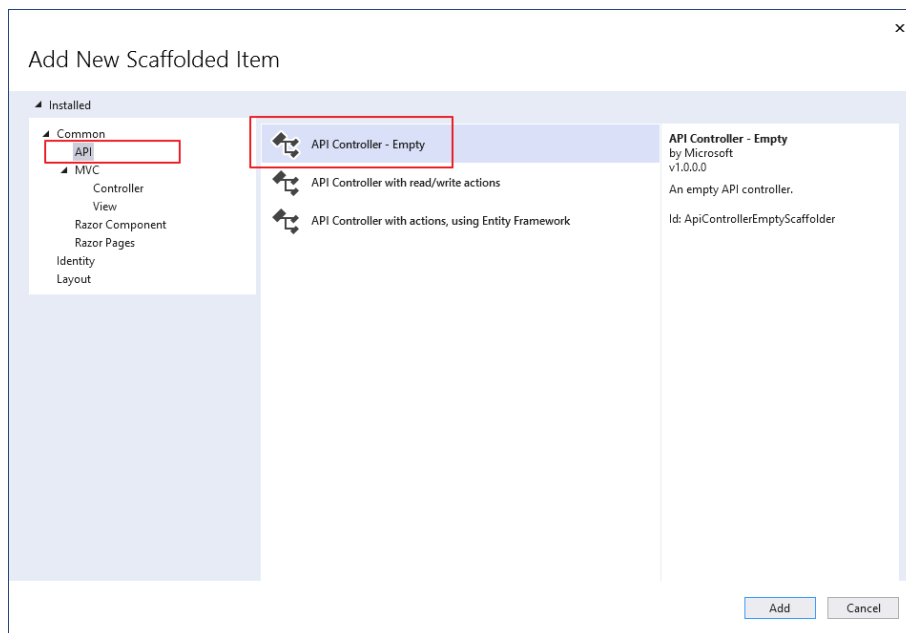
11 API CONTROLLERS TOEVOEGEN

We zullen nu de nodige API controllers voorzien die zullen dienst doen als API endpoints voor de clients die wensen te verbinden met onze API.

11.1 CATEGORIESCONTROLLER

Klik rechts op de folder **Controllers** en kies **Add → Controller...**

Kies in het volgende scherm voor **API** en daarna **API Controller - Empty**



Geef je nieuwe API controller de naam **CategoriesController**. Merk op dat we hier meevoud van Category gebruiken!

Onze controller erft over van **ControllerBase** en is reeds voorzien van enkele attributen:

```
using Microsoft.AspNetCore.Mvc;

namespace Pri.WebApi.Food.Api.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class CategoriesController : ControllerBase
    {
    }
}
```

```
[Route("api/[controller]")]
```

Het **Route** attribuut bevat een deel van de route, namelijk **api/** en daarna wordt de naam van de controller uitgelezen door `[controller]`, maar dan zonder de suffix **Controller**.

Het resultaat van dit route attribuut zal voor deze controller als volgt zijn: **api/categories**

```
[ApiController]
```

Het ApiController attribuut geeft aan dat een type en alle afgeleide typen worden gebruikt om HTTP API-responses te leveren.

Kortom, dit attribuut dienen we altijd te voorzien op onze Controller klassen indien deze als een API controller zullen gebruikt worden.

ControllerBase overerving

Is onze basis klasse voor onze API controller. Deze ControllerBase bevat geen support om Views te tonen, maar bevat enkel de basis functionaliteiten om deze controller voor een API te gebruiken.

11.1.1 SERVICE INJECTEREN

Voeg nu een constructor toe aan deze controller die ervoor zorgt dat de `ICategoryService` en `IProductService` interfaces worden geïnjecteerd.

```
public class CategoriesController : ControllerBase
{
    private readonly ICategoryService _categoryService;
    private readonly IProductService _productService;

    public CategoriesController(ICategoryService categoryService,
                               IProductService productService)
    {
        _categoryService = categoryService;
        _productService = productService;
    }
}
```

11.2 PRODUCTSCONTROLLER

Klik rechts op de folder **Controllers** en kies **Add → Controller...**

Kies in het volgende scherm voor **API** en daarna **API Controller - Empty**

Geef je nieuwe API controller de naam **ProductsController**. Merk op dat we hier meervoud van Product gebruiken!

Voorzie de DI voor de **IProductService**.

```
[Route("api/[controller]")]
[ApiController]
public class ProductsController : ControllerBase
{
    private readonly ICategoryService _categoryService;
    private readonly IProductService _productService;

    public ProductsController(ICategoryService categoryService,
                             IProductService productService)
    {
        _categoryService = categoryService;
        _productService = productService;
    }
}
```

11.3 SEARCHCONTROLLER

Klik rechts op de folder **Controllers** en kies **Add → Controller...**

Kies in het volgende scherm voor **API** en daarna **API Controller - Empty**

Geef je nieuwe API controller de naam **SearchController**.

Voorzie de DI voor de **IProductService**.

```
[Route("api/[controller]")]
[ApiController]
public class SearchController : ControllerBase
{
    private readonly IProductService _productService;

    public SearchController(IProductService productService)
    {
        _productService = productService;
    }
}
```


12 HTTP GET REQUESTS

We gaan nu de nodige action methods voorzien om collecties op te vragen van producten of categorieën en de action methods om een specifiek product of categorie op te vragen aan de hand van het id.

12.1 COLLECTIE VAN CATEGORIEËN OPVRAGEN

Voeg onderstaande ActionMethod toe in de CategoriesController:

```
[HttpGet]
public async Task<IActionResult> Get()
{
    var result = await _categoryService.ListAllAsync();
    if (result.Success)
    {
        var categoryDtos = result.Data.Select(c => new CategoryResponseDto
        {
            Id = c.Id,
            Name = c.Name
        }).ToList();
        return Ok(categoryDtos);
    }
    return BadRequest(result.Errors);
}
```

Deze methode is aan te roepen indien er een **GET** request gebeurt naar het **api/categories** endpoint.

Even alles overlopen!

[HttpGet]

Geeft aan dat de Get methode enkel met een HTTP GET request gebruikt kan worden

```
var result = await _categoryService.ListAllAsync();
```

We vragen alle categorieën op uit de database via onze CategoryService. Dit zal een ResultModel retourneren met de Success property die zal aangeven of de actie gelukt is. Indien de actie gelukt is zal de Data property de gevonden Categories bevatten.

```
if (result.Success)
{
    var categoryDtos = result.Data.Select(c => new CategoryResponseDto
    {
        Id = c.Id,
        Name = c.Name
    }).ToList();
    return Ok(categoryDtos);
}
```

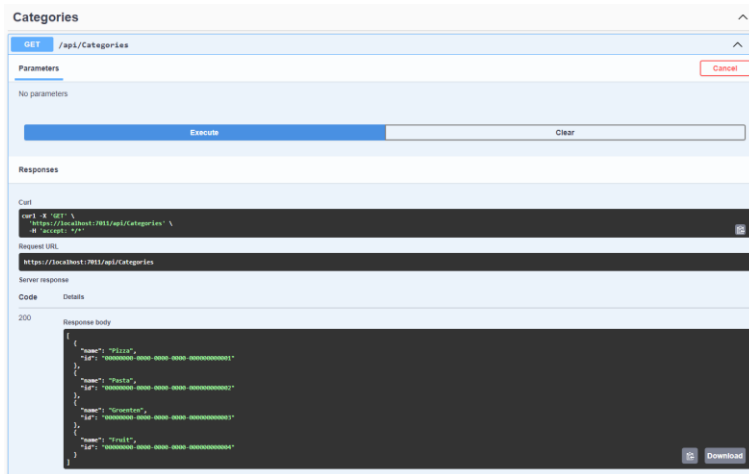
We gebruiken de Linq select methode om de categories om te zetten, te mapperen, naar een lijst van CategoryResponseDto's.

De lijst wordt vervolgens verstuurd naar de client via de Ok methode. Deze methode zal de client voorzien van een 200 response en zal de collectie omgezet worden naar een JSON object.

12.1.1 ENDPOINT UITTESTEN

Start je web API, en selecteer via de swagger pagina De Get endpoint `/api/categories`.

We zien dat we een collectie van alle categorieën terugkrijgen in een JSON object.



12.2 CATEGORIE OPVragen OP BASIS VAN ID

Voeg nu volgende methode toe aan de CategoriesController

```
[HttpGet("{id}")]
public async Task<IActionResult> Get(Guid id)
{
    if (await _categoryService.DoesCategoryIdExistsAsync(id) == false)
    {
        return NotFound();
    }
    var result = await _categoryService.GetByIdAsync(id);
    if (result.Success)
    {
        var categoryDto = new CategoryResponseDto
        {
            Id = result.Data.Id,
            Name = result.Data.Name
        };

        return Ok(categoryDto);
    }
    return BadRequest(result.Errors);
}
```

Deze methode wordt aangeroepen indien er een **GET** request gebeurt naar het **`api/categories/<id hier>`** endpoint.

Even overlopen!

```
[HttpGet("{id}")]
```

Geeft aan dat de Get methode enkel met een HTTP GET request gebruikt kan worden maar dat er een id moet worden uitgelezen in het endpoint. De naam van deze parameter moet overeenkomen met de naam van de parameter in de methode zelf.

```
public async Task<IActionResult> Get(Guid id)
```

De methode ontvangt een id parameter van het type Guid. Deze naam moet overeenkomen met de naam van de parameter in het HttpGet attribuut zodat dit kan uitgelezen worden uit het endpoint.

```
if (await _categoryService.DoesCategoryIdExistsAsync(id) == false)
{
    return NotFound();
}
```

We controleren eerst of de Category wel degelijk bestaat in de database en we retourneren een NotFound() indien nodig.

```
var result = await _categoryService.GetByIdAsync(id);
```

Vervolgens halen we de category op via de service klasse.

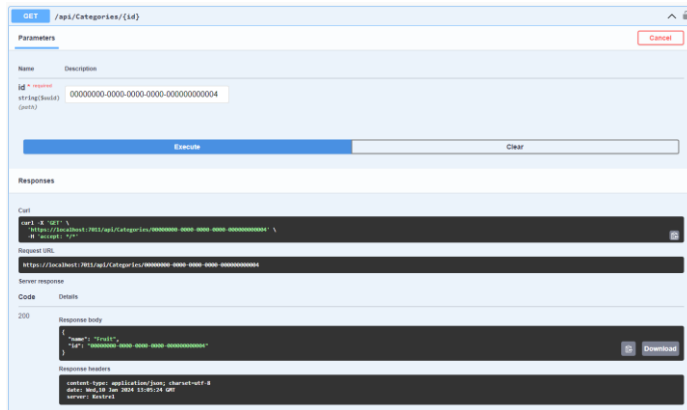
```
if (result.Success)
{
    var categoryDto = new CategoryResponseDto
    {
        Id = result.Data.Id,
        Name = result.Data.Name
    };
    return Ok(categoryDto);
}
```

Indien de Success property van het ResultModel object true is, wordt deze omgezet ("mappen") naar een object van het type CategoryResponseDto en dit retourneren we naar de client met de Ok methode die zorgt voor het terugsturen van de data met statuscode 200.

12.2.1 ENDPOINT UITTESTEN

Start de applicatie op en selecteer de /api/Categories/{id} endpoint. Geef als id de waarde 00000000-0000-0000-0000-000000000004 in.

We zien dat we de categorie met id 00000000-0000-0000-0000-000000000004 terugkrijgen in een JSON object.



12.3 COLLECTIE VAN PRODUCTEN OPVRAGEN

Voeg onderstaande ActionMethod toe in de ProductsController:

```
public async Task<IActionResult> Get()
{
    var result = await _productService.ListAllAsync();
    if (result.Success)
    {
        var productDtos = result.Data.Select(c => new ProductResponseDto
        {
            Id = c.Id,
            Name = c.Name,
            Category = new CategoryResponseDto
            {
                Id = c.Category.Id,
                Name = c.Category.Name
            },
            Image = $"{Request.Scheme}://{Request.Host}/img/{c.Image}"
        });
        return Ok(productDtos);
    }
    return BadRequest(result.Errors);
}
```

Deze methode wordt aangeroepen indien er een **GET** request gebeurt naar het **api/products** endpoint.

Even alles overlopen!

[HttpGet]

Geeft aan dat de Get methode enkel met een HTTP GET request gebruikt kan worden

```
var result = await _productService.ListAllAsync();
```

We vragen alle producten op uit de database via onze ProductService.

```
if (result.Success)
```

```

{
    var productDtos = result.Data.Select(c => new ProductResponseDto
    {
        Id = c.Id,
        Name = c.Name,
        Category = new CategoryResponseDto
        {
            Id = c.Category.Id,
            Name = c.Category.Name
        },
        Image = $"{Request.Scheme}://{Request.Host}/img/{c.Image}"
    });
    return Ok(productDtos);
}

```

We controleren de Success property van het ResultModel object om na te gaan of de actie gelukt is.

Via de **LINQ Select** methode vervolgens zal elk product (van het type **Product**) uit de collectie products (collectie van **Product uit de database**) omgezet worden naar een **nieuw ProductResponseDto** object en krijgen de properties **Id** en **Name** de waardes die in de Product entity aanwezig zijn.

We bouwen ook het pad naar de bijhorende image op:

```
Image = $"{Request.Scheme}://{Request.Host}/img/{c.Image}"
```

Request.Scheme bevat het scemah: https, en Request.Host bevat de naam van onze host: localhost

Aangezien er ook aan Category van het type **CategoryResponseDto** aanwezig moet zijn in elk **ProductResponseDto** object maken we hiervoor ook een **nieuw** object aan van het type **CategoryResponseDto**.

Alle omgezette klassen van Product naar ProductResponseDto zullen bewaard worden in de collectie van productsDto.

```
return Ok(productsDto);
```

De collectie van ProductResponseDto objecten zal verstuurd worden naar de client via de Ok methode. Deze methode zal de client voorzien van een 200 response en zal de collectie omgezet worden naar een JSON object.

In het geval dat de Success property false bevat, retourneren we een BadRequest() met de bijhorende Errors.

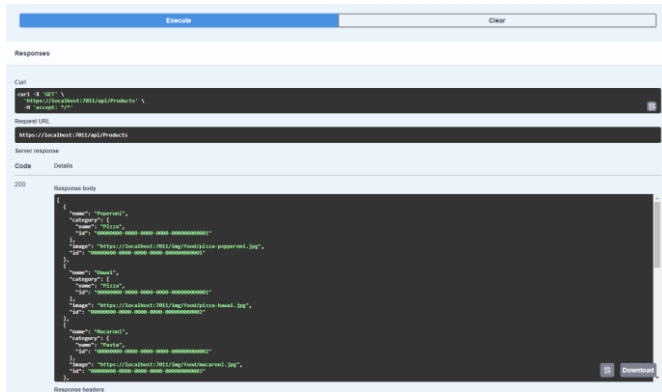
```
return BadRequest(result.Errors);
```

12.3.1 ENDPOINT UITTESTEN

Start de applicatie en selecteer het endpoint /api/products:

<https://localhost:5001/api/products>

We zien dat we een collectie van alle producten terugkrijgen in een JSON object. Merk op dat ook de bijhorende categorie aanwezig is, alsook de bijhorende afbeelding.



12.4 PRODUCT OPVRAGEN OP BASIS VAN ID

Voeg nu volgende methode toe aan de ProductsController

```
[HttpGet("{id}")]
public async Task<IActionResult> Get(Guid id)
{
    if (await _categoryService.DoesCategoryIdExistsAsync(id) == false)
    {
        return NotFound();
    }
    var result = await _productService.GetByIdAsync(id);
    if (result.Success)
    {
        var productDto = new ProductResponseDto
        {
            Id = result.Data.Id,
            Name = result.Data.Name,
            Category = new CategoryResponseDto
            {
                Id = result.Data.Category.Id,
                Name = result.Data.Category.Name
            },
            Image = $"{Request.Scheme}://{Request.Host}/img/{result.Data.Image}"
        };
        return Ok(productDto);
    }
    return BadRequest(result.Errors);
}
```

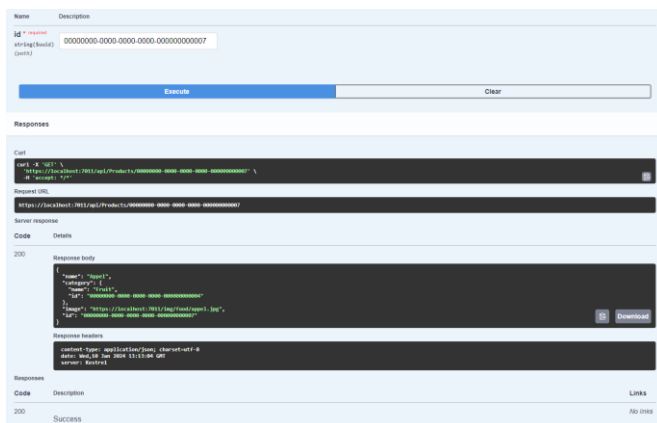
Deze methode wordt aangeroepen indien er een **GET** request gebeurt naar het **api/products/<id hier>** endpoint. Merk op dat er eerst gecontroleerd wordt of het product bestaat aan de hand van de methode *DoesProductIdExistsAsync()*. Vervolgens wordt gelijkaardige code gebruikt aan de vorige Get methode om een ProductDto op te bouwen en terug te sturen naar de client.

12.4.1 ENDPOINT UITTESTEN

Start de applicatie en selecteer het endpoint `/api/products/{id}`. Geef als id de volgende waarde in:

00000000-0000-0000-0000-000000000007

We zien dat we het product met id 00000000-0000-0000-0000-000000000007 terugkrijgen in een JSON object



12.5 ALLE PRODUCTEN VAN ÉÉN CATEGORIE OPVRAGEN OP BASIS VAN ID

De ProductService gebruiken we nog niet in de CategoriesController, maar deze zal nu van pas komen.

Voeg de volgende methode toe aan de CategoriesController:

```
[HttpGet("{id}/products")]
public async Task<IActionResult> GetProductsFromCategory(Guid id)
{
    if (await _categoryService.DoesCategoryIdExistsAsync(id) == false)
    {
        return NotFound($"No category with an id of {id}");
    }
    var productsResult = await _productService.GetByCategoryIdAsync(id);
    if (productsResult.Success)
    {
        var productsDto = productsResult.Data.Select(p => new ProductResponseDto
        {
            Id = p.Id,
            Name = p.Name,
            Image = $"{Request.Scheme}://{Request.Host}/img/{p.Image}",
            Category = new CategoryResponseDto
            {
                Id = p.Category.Id,
                Name = p.Category.Name
            }
        });
    }
}
```

```

    }
    }).ToList();
    return Ok(productsDto);
}
return BadRequest(productsResult.Errors);
}

```

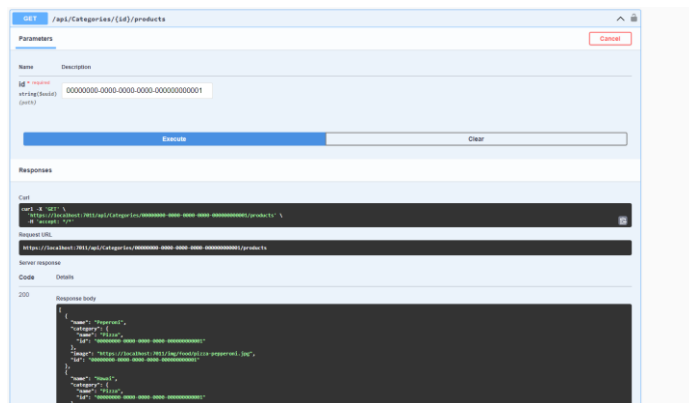
Deze wordt aangeroepen indien er een **GET** request gebeurt naar het **api/categories/<id van category hier>/products** endpoint.

ENDPOINT UITTESTEN

Start de applicatie en selecteer het endpoint: **/api/categories/{id}/products**

Geef id 00000000-0000-0000-0000-000000000001 in.

We zien dat we alle producten terugkrijgen die behoren tot de categorie met ID 00000000-0000-0000-0000-000000000001 in een JSON object.



12.6 ZOEKEN OP CATEGORIE EN/OF PRODUCT

Voeg onderstaande ActionMethod toe in de SearchController:

```

[HttpGet]
public async Task<IActionResult> Search([FromQuery] string searchQuery)
{
    var searchResult = await _productService.SearchAsync(searchQuery);
    if (searchResult.Success)
    {
        var searchResultsDto = searchResult.Data.Select(s => new ProductResponseDto
        {
            Id = s.Id,
            Name = s.Name,
            Category = new CategoryResponseDto
            {
                Id = s.Category.Id,
                Name = s.Category.Name
            }
        });
    }
}

```



```

    }
    });
    return Ok(searchResultsDto);
}
return BadRequest(searchResult.Success);
}

```

Deze methode wordt aangeroepen indien er een **GET** request gebeurt naar het **api/search?searchQuery=<zoekterm>** endpoint.

Even alles overlopen!

```
public async Task<ActionResult> Search([FromQuery] string searchQuery)
```

Het FromQuery attribuut heeft aan dat er op zoek gegaan moet worden naar een queryparameter met de naam searchQuery.

```
var searchResults = await _productService.SearchAsync(searchQuery);
```

We vragen alle producten op uit de database waarvan de naam of de categorie naam een deel van de searchQuery bevat via de SearchAsync methode van de ProductService.

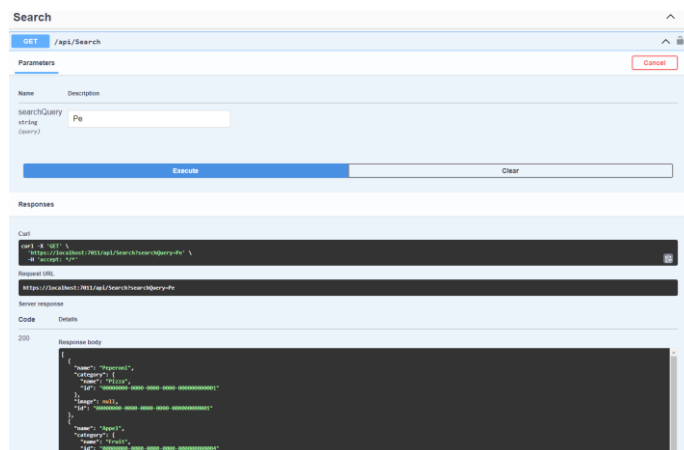
Vervolgens controleren we de Success property om, indien true, de lijst van gevonden producten om te zetten naar een lijst van ProductResponseDto's.

12.6.1 ENDPOINT UITTESTEN

Start de applicatie en selecteer het endpoint /api/Search. Geef als zoekterm 'Pe' in om alle producten met 'pe' in de naam op te halen.

<https://localhost:5001/api/search?searchQuery=pe>

We zien dat we een collectie van alle producten terugkrijgen in een JSON object die voldoen aan de zoekactie



13 HTTP POST REQUESTS

We gaan nu de nodige action methods voorzien om categorieën en producten toe te voegen. Hiervoor zullen we nieuwe DTO klassen aanmaken.

13.1 CATEGORYREQUESTDTO AANMAKEN

Maak in de folder Dtos/Categories een nieuwe klasse aan met de naam CategoryRequestDto een voorzie volgende code:

```
public class CategoryRequestDto: BaseDto
{
    [Required(ErrorMessage = "{0} is required")]
    public string Name { get; set; }
}
```

Merk op dat we hier een Required attribuut op de property Name hebben geplaatst. Hierdoor dwingen we af dat de Name property verplicht is voor de client.

13.2 PRODUCTREQUESTDTO AANMAKEN

Maak in de folder Dtos/Products een nieuwe klasse aan met de naam ProductRequestDto een voorzie volgende code:

```
public class ProductRequestDto : BaseDto
{
    [Required(ErrorMessage = "{0} is required")]
    public string Name { get; set; }

    [Required(ErrorMessage = "{0} is required")]
    public Guid CategoryId { get; set; }
}
```

Merk op dat we hier een Required attribuut op de property Name en CategoryId hebben geplaatst. Hierdoor dwingen we af dat de Name en CategoryId property verplicht is voor de client.

13.3 CATEGORIE TOEVOEGEN

Voeg onderstaande ActionMethod toe in de CategoriesController:

```
[HttpPost]
public async Task<IActionResult> Add(CategoryRequestDto categoryDto)
{
    Category category = new Category
    {
        Name = categoryDto.Name
    };
    var result = await _categoryService.AddAsync(category);
    if (result.Success)
    {
        var dto = new CategoryResponseDto
        {
            Id = category.Id,
            Name = category.Name
        };
        return CreatedAtAction(nameof(Get), new { id = category.Id }, dto);
    }
    return BadRequest(result.Errors);
}
```

Deze methode is aan te roepen indien er een **POST** request gebeurt naar het [api/categories](#) endpoint met een object van het type CategoryRequestDto in de body van je request.

Even alles overlopen!

```
[HttpPost]
```

Geeft aan dat de Add methode enkel met een HTTP POST request gebruikt kan worden

```
Category category = new Category
{
    Name = categoryDto.Name
};
```

We maken een nieuw Category object aan op basis van het binnenkomende categoryDto.

```
var result = await _categoryService.AddAsync(category);
if (result.Success)
{
    var dto = new CategoryResponseDto
    {
        Id = category.Id,
        Name = category.Name
    };
    return CreatedAtAction(nameof(Get), new { id = category.Id }, dto);
}
```

Vervolgens geven we het object mee aan de AddAsync methode van de CategoryService. Het antwoord zal een ResultModel object zijn waarvan we vervolgens de Success property gaan testen. Indien die true is, retourneren we een CategoryResponseDto met de net aangemaakte Category terug.

```
return CreatedAtAction(nameof(Get), new { id = category.Id }, dto);
```

We versturen een 201 (Created) response code naar de client om te laten weten dat het toevoegen succesvol is verlopen. In ons geval sturen we geen object terug naar de client. De parameters zorgen ervoor dat in de response de url waar de resource opgevraagd kan worden, gegenereerd wordt. Dat gebeurt in de Location header van de response.

13.3.1 ENDPOINT UITTESTEN MET POSTMAN

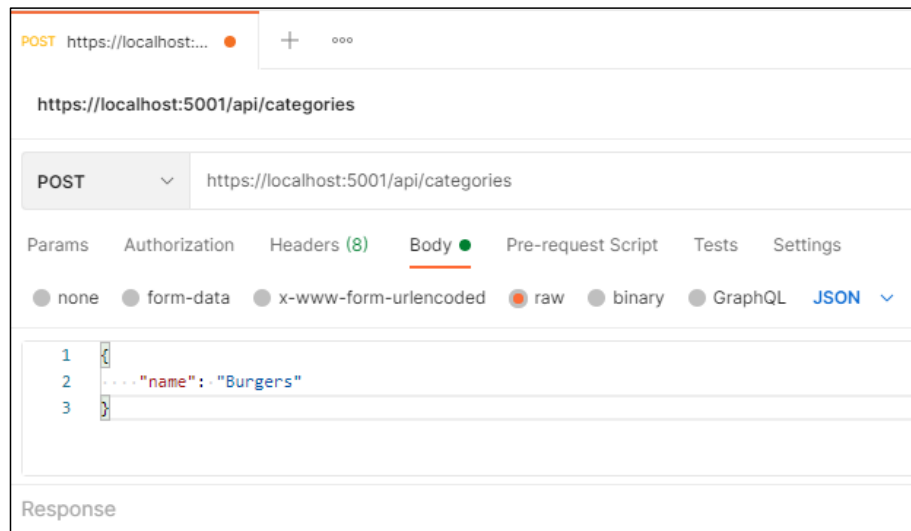
Deze endpoint gaan we testen met Postman om een beter zicht te krijgen op het antwoord van de api. Je Postman is een http client waarmee we http requests naar Api's kunnen sturen. Het wordt voornamelijk gebruikt om API's te testen. De downloadlink vind je [hier](#).

Open dus Postman en doe nu een POST request naar volgend endpoint:

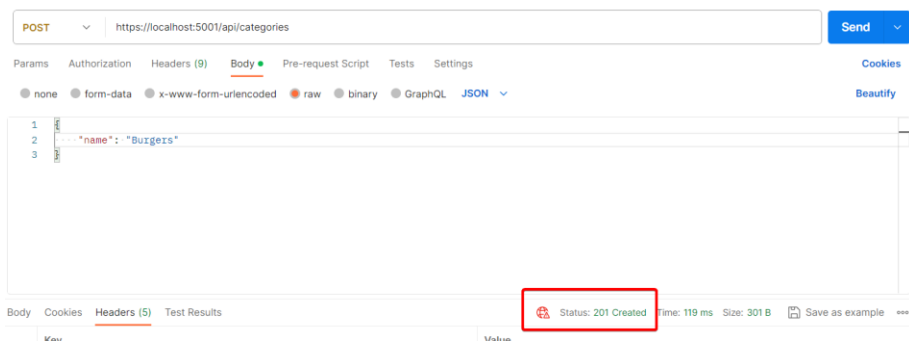
<https://localhost:5001/api/categories>

Voeg in de **body** van je **request** volgende json toe:

```
{
  "name": "Burgers"
}
```

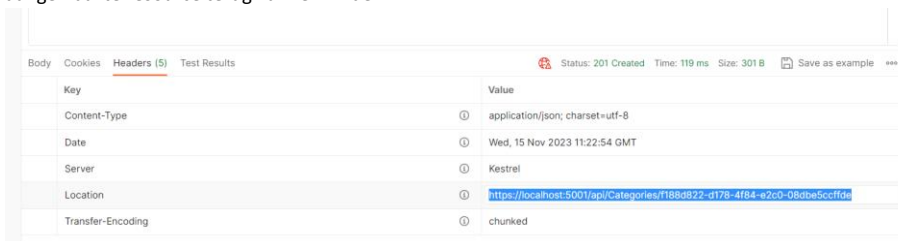


Druk nu op Send om de POST request met het JSON object in de body door te sturen naar onze WEB API:



We krijgen in de body van de response het object terug (want de `CreatedAtAction()` methode van het endpoint stuurt het aangemaakte object terug als DTO). Bovendien zien we dat het toevoegen van een nieuwe categorie gelukt is aan de statuscode in de response → 201 Created

Onder headers vinden we ook de 'Location' header waar we het resultaat terugvinden van de URL die opgebouwd wordt door de `CreatedAtAction`-methode. De URL is de plaats waar we de zonet aangemaakte resource terug kunnen vinden:



Test ook opnieuw even het endpoint uit om alle categorieën op te vragen:

GET /api/categories

We zien in de response van de collectie dat de categorie met naam Burgers werd toegevoegd. Uiteraard zal de ID bij jullie er anders uitzien omdat die automatisch aangemaakt wordt door de database.

```
[
  {
    "name": "Pizza",
    "id": "00000000-0000-0000-0000-000000000001"
  },
  {
    "name": "Pasta",
    "id": "00000000-0000-0000-0000-000000000002"
  },
  {
    "name": "Groenten",
    "id": "00000000-0000-0000-0000-000000000003"
  },
  {
    "name": "Fruit",
    "id": "00000000-0000-0000-0000-000000000004"
  },
  {
    "name": "Burgers",
    "id": "f188d822-d178-4f84-e2c0-08dbe5ccffde"
  }
]
```

13.4 PRODUCT TOEVOEGEN

Voeg onderstaande ActionMethod toe in de ProductsController:

```
[HttpPost]
public async Task<IActionResult> Add(ProductRequestDto productDto)
{
    if (!await _categoryService.DoesCategoryIdExistsAsync(productDto.CategoryId))
    {
        return BadRequest($"Cannot add new product because category with id {product
Dto.CategoryId} does not exists");
    }
    var product = new Product
    {
        CategoryId = productDto.CategoryId,
        Name = productDto.Name,
        Image = "food/default.jpg"
    };
    var resultProduct = await _productService.AddAsync(product);
    if (resultProduct.Success)
    {
        var resultCategory = await _categoryService.GetByIdAsync(product.CategoryId);
        if (resultCategory.Success)
        {
            var dto = new ProductResponseDto
            {
                Id = product.Id,
                Name = product.Name,
                Category = new CategoryResponseDto
                {
                    Id = resultCategory.Data.Id,
                    Name = resultCategory.Data.Name
                },
                Image = $"{Request.Scheme}://{Request.Host}/img/{product.Image}"
            };
            return CreatedAtAction(nameof(Get), new { id = product.Id }, dto);
        }
        return BadRequest(resultCategory.Errors);
    }
    return BadRequest(resultProduct.Errors);
}
```

Deze methode wordt aangeroepen indien er een **POST** request gebeurt naar het **api/products** endpoint met een object van het type ProductRequestDto in de body van je request.

Aangezien we hier eerder de CategoryService injecteerden kunnen we deze nu ook voor de eerste keer gebruiken.

Even alles overlopen!

[HttpPost]

Geeft aan dat de Add methode enkel met een HTTP POST request gebruikt kan worden

```

if (await _categoryService.DoesCategoryIdExistsAsync(productDto.CategoryId))
{
    return BadRequest($"Cannot add new product because category with id
    {product dto.CategoryId} does not exists");
}

```

Wanneer we een product willen toevoegen, dan moeten we onmiddellijk ook een categorie eraan koppelen. Dat doet een client door het id van een categorie mee te geven in de DTO. Het is echter mogelijk dat een client fouten maakt en bijvoorbeeld een niet-bestaande categorie wil koppelen. Daarom checken we eerst, m.b.v. de categoryService, of de categorie die de client wil koppelen wel degelijk bestaat. Indien dat niet zo zou zijn, dan wordt dit kenbaar gemaakt worden aan de clients door de BadRequest methode.

```

var product = new Product
{
    CategoryId = productDto.CategoryId,
    Name = productDto.Name,
    Image = "food/default.jpg";
}

```

De productDto wordt gemapped naar een Product entity. Merk op dat we een default waarde ingeven voor de image property. In het project is er een default.jpg aanwezig hiervoor. Later komt het effectief uploaden van een afbeelding aan bod.

```

var resultProduct = await _productService.AddAsync(product);
if (resultProduct.Success)
{
    var resultCategory = await _categoryService.GetByIdAsync(product.CategoryId);
    if (resultCategory.Success)
    {
        var dto = new ProductResponseDto
        {
            Id = product.Id,
            Name = product.Name,
            Category = new CategoryResponseDto
            {
                Id = resultCategory.Data.Id,
                Name = resultCategory.Data.Name
            },
            Image = $"{Request.Scheme}://{Request.Host}/img/{product.Image}"
        };
        return CreatedAtAction(nameof(Get), new { id = product.Id }, dto);
    }
    return BadRequest(resultCategory.Errors);
}
return BadRequest(resultProduct.Errors);
}

```

We voegen het product toe aan de database via de AddAsync methode van de ProductService. We gebruiken de resultProduct.Success property om na te gaan of het toevoegen gelukt is. Vervolgens halen we de category op om die te kunnen toevoegen aan het Dto object met het nieuwe product die we gaan

terugsturen door middel van de CreatedAtAction methode. Hier wordt ook gecontroleerd of de category succesvol opgehaald werd.

```
return CreatedAtAction(nameof(Get), new { id = product.Id }, dto);
```

We versturen een 201 (Created) response code naar de client om te laten weten dat het toevoegen succesvol is verlopen. In ons geval sturen we geen object terug naar de client. De parameters zorgen ervoor dat in de response de url waar de resource opgevraagd kan worden, gegenereerd wordt. Dat gebeurt in de Location header van de response.

13.4.1 ENDPOINT UITTESTEN MET POSTMAN

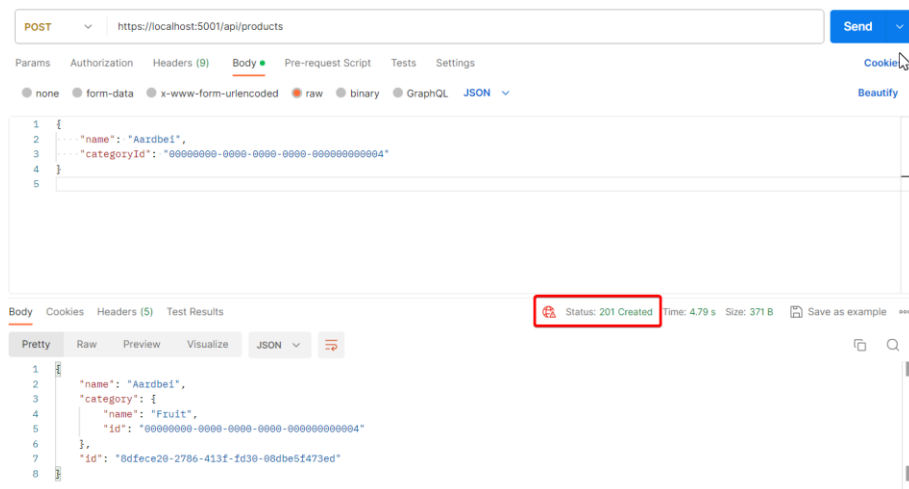
Open Postman en doe nu een POST request naar volgend endpoint:

<https://localhost:5001/api/products>

Voeg in de **body** van je **request** volgende json toe:

```
{
  "name": "Aardbei",
  "categoryId": "00000000-0000-0000-0000-000000000004"
}
```

Druk nu op Send om de POST request met het JSON object in de body door te sturen naar onze WEB API:



We krijgen in de body van de response het object terug (want de CreatedAtAction() methode van het endpoint stuurt het aangemaakte object terug als DTO). Bovendien zien we dat het toevoegen van een nieuwe categorie gelukt is aan de statuscode in de response → 201 Created

Onder headers vinden we ook de 'Location' header waar we het resultaat terugvinden van de URL die opgebouwd wordt door de CreatedAtAction-methode. De URL is de plaats waar we de zonet aangemaakte resource terug kunnen vinden:

Body	Cookies	Headers (5)	Test Results	Status: 201 Created	Time: 4.79 s	Size: 371 B	Save as example	***
Key		Value						
Content-Type		①	application/json; charset=utf-8					
Date		①	Wed, 15 Nov 2023 16:03:47 GMT					
Server		①	Kestrel					
Location		①	https://localhost:5001/api/Products/8df6ce20-2786-413f-fd30-06d6e3f473ec					
Transfer-Encoding		①	chunked					

GET /api/products

We zien in de response van de collectie dat het product met naam Aardbei werd toegevoegd. Uiteraard zal het ID bij jullie er anders uitzien.

```
[
  ...
  {
    "name": "Tomaat",
    "category": {
      "name": "Groenten",
      "id": "00000000-0000-0000-0000-000000000003"
    },
    "id": "00000000-0000-0000-0000-000000000006"
  },
  {
    "name": "Appel",
    "category": {
      "name": "Fruit",
      "id": "00000000-0000-0000-0000-000000000004"
    },
    "id": "00000000-0000-0000-0000-000000000007"
  },
  {
    "name": "Peer",
    "category": {
      "name": "Fruit",
      "id": "00000000-0000-0000-0000-000000000004"
    },
    "id": "00000000-0000-0000-0000-000000000008"
  },
  {
    "name": "Aardbei",
    "category": {
      "name": "Fruit",
      "id": "00000000-0000-0000-0000-000000000004"
    },
    "id": "8dfece20-2786-413f-fd30-08dbe5f473ed"
  }
]
```

14 HTTP PUT REQUESTS

We gaan nu de nodige action methods voorzien om producten of categorieën aan te passen.

14.1 CATEGORIE AANPASSEN

Voeg onderstaande ActionMethod toe in de CategoriesController:

```
[HttpPut]
public async Task<IActionResult> Update(CategoryRequestDto categoryDto)
{
    if (await _categoryService.DoesCategoryIdExistsAsync(categoryDto.Id) == false)
    {
        return BadRequest($"No category with id '{categoryDto.Id}' found");
    }
    var existingCategoryResult = await _categoryService.GetByIdAsync(categoryDto.Id);
    if (existingCategoryResult.Success == false)
    {
        return BadRequest(existingCategoryResult.Errors);
    }
    var existingEntity = existingCategoryResult.Data;
    existingEntity.Name = categoryDto.Name;
    var result = await _categoryService.UpdateAsync(existingEntity);
    if (result.Success)
    {
        return Ok($"Category {existingEntity.Id} updated");
    }
    return BadRequest(result.Errors);
}
```

Deze methode wordt aangeroepen indien er een **PUT** request gebeurt naar het **api/categories** endpoint met een object van het type CategoryRequestDto in de body van je request.

In bovenstaande code gaan we eerst kijken of de category die we willen updaten wel een gekende category is door de DoesCategoryIdExistsAsync methode van de CategoryService aan te roepen. Indien de client een category wil updaten die niet bestaat in onze applicatie, dan sturen we een BadRequest (statuscode 400) terug.

Bestaat de categorie wel, dan halen we de bestaande Category op. We kunnen nu de waardes uit de DTO (die de updates bevatten) toekennen aan de existingCategory Entity.

```
var existingEntity = existingCategoryResult.Data;
```

Nu we een de Category entity hebben, kunnen we deze doorspelen aan de CategoryService UpdateAsync-methode, die de category zal updaten.

```
var result = await _categoryService.UpdateAsync(existingEntity);
if (result.Success)
{
    return Ok($"Category {existingEntity.Id} updated");
}
return BadRequest(result.Errors);
```

Indien er fouten gebeurden (hier: de categorienaam bestaat al, zie de methode in de service), dan sturen we wederom een BadRequest terug, ditmaal met een lijst van alle foutmeldingen als body.

We sturen vervolgens een Ok-object (statuscode 200) terug met een (optionele) boodschap.

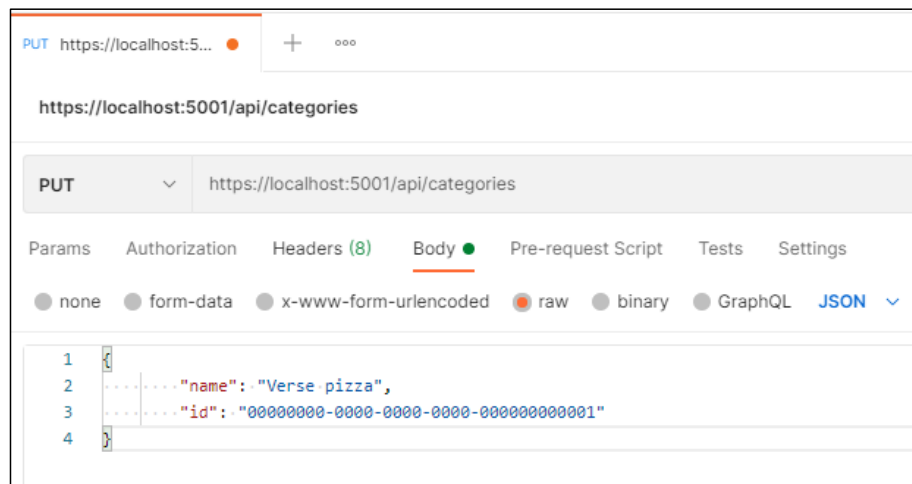
14.1.1 ENDPOINT UITTESTEN MET POSTMAN

Open Postman en doe nu een PUT request naar volgend endpoint:

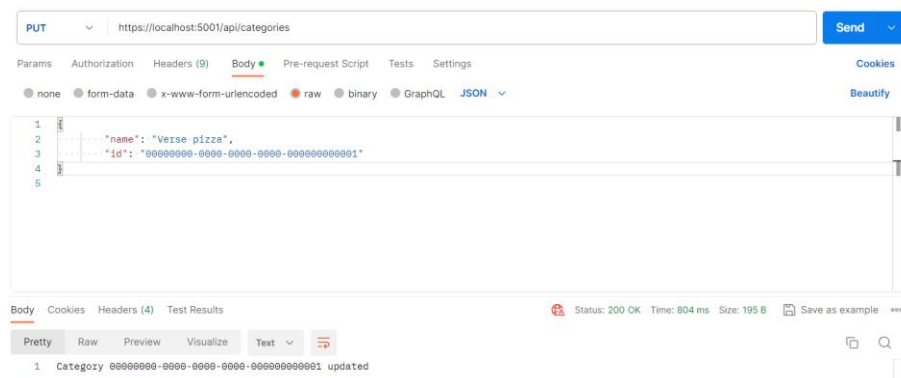
<https://localhost:5001/api/categories>

Voeg in de **body** van je **request** volgende json toe:

```
{
  "name": "Verse pizza",
  "id": "00000000-0000-0000-0000-000000000001"
}
```



Druk nu op Send om de PUT request met het JSON object in de body door te sturen naar onze WEB API:



Test ook opnieuw even het endpoint uit om alle categorieën op te vragen:

GET /api/categories

We zien in de response van de collectie dat de categorie met naam Pizza aangepast werd naar 'Verse pizza'.

```
[
  {
    "name": "Verse pizza",
    "id": "00000000-0000-0000-0000-000000000001"
  },
  {
    "name": "Pasta",
    "id": "00000000-0000-0000-0000-000000000002"
  },
  {
    "name": "Groenten",
    "id": "00000000-0000-0000-0000-000000000003"
  },
  {
    "name": "Fruit",
    "id": "00000000-0000-0000-0000-000000000004"
  },
  {
    "name": "Burgers",
    "id": "17112c71-8e95-4c4b-f9d4-08d986851b2e"
  }
]
```

Probeer ook de update te herhalen, maar update ditmaal met naar een bestaande categorie, nl. de 'burgers' die we eerder aanmaakten. Je krijgt nu een array met foutboodschap(en) terug omdat de update die we willen doorvoeren zou betekenen dat we 2x een record bijhouden met dezelfde naam (de categorie bestaat al onder een ander id). We kiezen ervoor om dit niet toe te laten; categorieën moeten uniek zijn:

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** https://localhost:5001/api/categories
- Body:** A JSON object with "name": "Burgers" and "id": "00000000-0000-0000-0000-000000000001".
- Response:** Status 400 Bad Request, Time: 16 ms, Size: 201 B. The response body is: "A category with this name already exists".

14.2 PRODUCT AANPASSEN

Voeg onderstaande ActionMethod toe in de ProductsController:

```
[HttpPut]
public async Task<IActionResult> Update(ProductRequestDto productDto)
{
    if (await _productService.DoesProductIdExistAsync(productDto.Id) == false)
    {
        return BadRequest($"No product with id '{productDto.Id}' found");
    }
    var existingProductResult = await _productService.GetByIdAsync(productDto.Id);
    if (existingProductResult.Success == false)
    {
        return BadRequest(existingProductResult.Errors);
    }
    var existingEntity = existingProductResult.Data;
    existingEntity.CategoryId = productDto.CategoryId;
    existingEntity.Name = productDto.Name;
    var result = await _productService.UpdateAsync(existingEntity);
    if (result.Success)
    {
        return Ok($"Product {existingEntity.Id} updated");
    }
    return BadRequest(result.Errors);
}
```

Deze methode wordt aangeroepen indien er een **PUT** request gebeurt naar het **api/products** endpoint met een object van het type ProductRequestDto in de body van je request.

De methode checkt eerst of het product wel degelijk bestaat door middel van de DoesProductIdExistAsync methode.

Als het product bestaat halen we het op en we passen de properties aan.

Vervolgens voeren we de update door m.b.v. de UpdateAsync()-methode van de productService. Indien dit slaagt, sturen we een Ok object terug.

Let wel: Vooraleer de update effectief door te voeren naar de database dienen we wel nog een extra controle uitvoeren op het al dan niet bestaan van de CategoryId. We kunnen een product immers niet updaten naar een onbestaande categorie. Plaats onderstaande code in de ProductService klasse, in de UpdateAsync() methode, net voor het updaten van de entity.

```
//does category id exist
if(!await _applicationDbContext.Categories
    .AnyAsync(c => c.Id.Equals(entity.CategoryId)))
{
    return new ResultModel<Product>
    {
        Errors = new List<string> { $"Category id:{entity.CategoryId} does not exist!" }
    };
}
```

We gebruiken de AnyAsync() Linq methode op de Categories om na te gaan of de category wel degelijk bestaat. Indien niet, retourneren we een ResultModel object met de gepaste foutmelding. Merk ook op

dat deze controle ook in de api methode uitgevoerd kan worden, door gebruik te maken van de DoesCategoryIdExists() methode van de CategoryService klasse.

14.2.1 ENDPOINT UITTESTEN MET POSTMAN

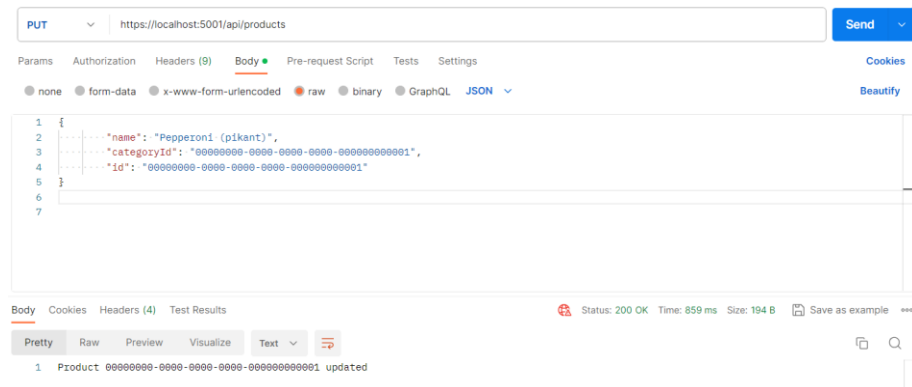
Open Postman en doe nu een PUT request naar volgend endpoint:

<https://localhost:5001/api/products>

Voeg in de **body** van je **request** volgende json toe:

```
{
  "name": "Pepperoni (pikant)",
  "categoryId": "00000000-0000-0000-0000-000000000001",
  "id": "00000000-0000-0000-0000-000000000001"
}
```

Druk nu op Send om de PUT request met het JSON object in de body door te sturen naar onze WEB API:



We krijgen een response met statuscode 200 OK terug, samen met de boodschap dat de update slaagde.

Test ook opnieuw even het endpoint uit het product met het id op te vragen:

GET /api/products/00000000-0000-0000-0000-000000000001

We zien in de response dat de naam Pepperoni aangepast werd naar Pepperoni (pikant).

```
{
  "name": "Peperoni (pikant)",
  "category": {
    "name": "Verse pizza",
    "id": "00000000-0000-0000-0000-000000000001"
  },
  "id": "00000000-0000-0000-0000-000000000001"
}
```

15 HTTP DELETE REQUESTS

We gaan nu de nodige action methods voorzien om producten of categorieën te verwijderen.

15.1 CATEGORIE VERWIJDEREN

Voeg onderstaande ActionMethod toe in de CategoriesController:

```
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(Guid id)
{
    if (await _categoryService.DoesCategoryIdExistsAsync(id) == false)
    {
        return NotFound($"No category with an id of {id}");
    }
    var existingCategoryResult = await _categoryService.GetByIdAsync(id);

    var result = await _categoryService.DeleteAsync(existingCategoryResult.Data);
    if (result.Success)
    {
        return Ok($"Category {existingCategoryResult.Data.Id} deleted");
    }
    return BadRequest(result.Errors);
}
```

Let wel: ook hier gaan we een extra check voorzien in onze ProductService klasse om na te gaan of een categorie geen producten verbonden heeft, de database zal immers niet toelaten dat we de categorie verwijderen in dat geval.

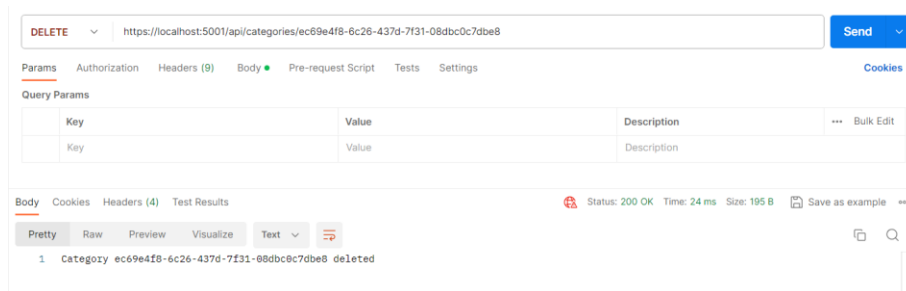
Plaats onderstaande code in de CategoryService methode DeleteAsync(), net voor het aanroepen van de remove methode van de ApplicationDbContext.

Deze methode is aan te roepen indien er een **DELETE** request gebeurt naar het **api/categories/<id van category>** endpoint.

15.1.1 ENDPOINT UITTESTEN MET POSTMAN

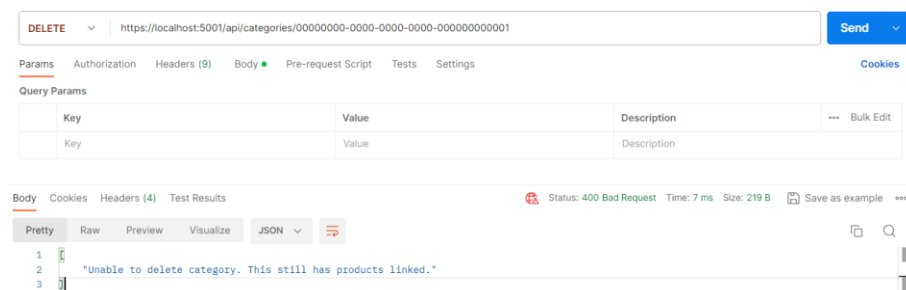
Open Postman en maak eerst een nieuwe categorie aan die we straks zullen verwijderen. Doe nu een DELETE request naar volgend endpoint:

<https://localhost:5001/api/categories/<id-van-je-te-verwijderen-categorie>>



Uit de responsecode (statuscode 200 OK) leren we dat het verwijderen van de categorie lukte. Bovendien krijgen we een aanvullende boodschap meegestuurd in de response body.

Willen we een categorie verwijderen die producten heeft gelinkt, zoals bijvoorbeeld categorie met (verse) pizza's, dan krijgen we in de BadRequest response (statuscode 400) terug met een array van foutboodschap(en):



15.2 PRODUCT VERWIJDEREN

Voeg onderstaande ActionMethod toe in de ProductsController:

```
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(Guid id)
{
    if (await _productService.DoesProductIdExistsAsync(id) == false)
    {
        return NotFound($"No product with an id of {id}");
    }
    var existingProductResult = await _productService.GetByIdAsync(id);
    if (existingProductResult.Success == false)
    {
        return BadRequest(existingProductResult.Errors);
    }
    var result = await _productService.DeleteAsync(existingProductResult.Data);
    return Ok($"Product {existingProductResult.Data.Id} deleted");
}
```

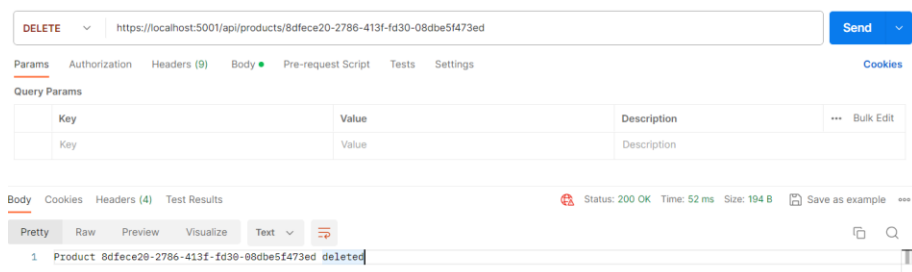
Deze methode wordt aangeroepen indien er een **DELETE** request gebeurt naar het **api/products/<id van product>** endpoint.

Gezien een product geen afhankelijkheden heeft, is delete code heel wat eenvoudiger: indien het product bestaat, dan verwijderen we het.

ENDPOINT UITTESTEN MET POSTMAN

Open Postman en maak eerst een nieuw product aan die we straks zullen verwijderen. Doe nu een DELETE request naar volgend endpoint:

<https://localhost:5001/api/products/<id-van-je-te-verwijderen-product>>



Uit de responsecode (statuscode 200 OK) leren we dat het verwijderen van de categorie lukte. Bovendien krijgen we een aanvullende boodschap meegestuurd in de response body.

16 EEN FILE UPLOADEN

Vaak bestaat de noodzaak om een bestand te laten uploaden naar een webapplicatie. In de module Web Backend zagen we reeds hoe dit geïmplementeerd kan worden. In een Api project komt dit grotendeels op dezelfde werkwijze neer. Enig verschil is in de manier hoe de clients de gegevens van een product naar de Web api moeten doorsturen. Met Json kan men immers geen bestanden doorsturen, en dus zal er door een client applicatie met form data moeten worden gewerkt.

In onze voorbeeldapplicatie gaan we een extra api methode toevoegen aan de ProductsController die zal toelaten dat een nieuw product aangemaakt kan worden met een image upload.

We maken eerst een Interface aan IImageService, met één methode die de upload voor zijn rekening zal nemen. Plaats deze file in het Core project => Services => Interfaces folder.

```
public interface IImageService
{
    Task<ResultModel<string>> AddOrUpdateImageAsync(IFormFile image, string fileName = "");
}
```

Deze methode ontvangt een IFormFile image en een optionele fileName om een fileUpdate te kunnen uitvoeren.

Maak nu de implementatieklasse ImageService aan in de folder Services van het Core project en plaats er onderstaande code in.

```
public class ImageService : IImageService
{
    private readonly IHostEnvironment _hostEnvironment;

    public ImageService(IHostEnvironment hostEnvironment)
    {
        _hostEnvironment = hostEnvironment;
    }

    public async Task<ResultModel<string>> AddOrUpdateImageAsync(
        IFormFile image, string fileName = "")
    {
        if (fileName == "")
        {
            fileName = $"{Guid.NewGuid()}_{Path.GetExtension(image.FileName)}";
        }

        var pathOnDisk = Path.Combine(_hostEnvironment.ContentRootPath, "wwwroot",
            "img", "food");

        if (!Directory.Exists(pathOnDisk))
        {
            Directory.CreateDirectory(pathOnDisk);
        }

        var completePathWithFilename = Path.Combine(pathOnDisk, fileName);
```

```

using (FileStream fileStream = new(completePathWithFilename, FileMode.Create))
{
    try
    {
        await image.CopyToAsync(fileStream);
        return new ResultModel<string>
        {
            Data = fileName
        };
    }
    catch (FileNotFoundException exception)
    {
        return new ResultModel<string>
        {
            Errors = new List<string> { exception.Message }
        };
    }
}
}
}

```

We bekijken deze code nader.

We declareren en injecteren de interface IHostEnvironment:

```

private readonly IHostEnvironment _hostEnvironment;

public ImageService(IHostEnvironment hostEnvironment)
{
    _hostEnvironment = hostEnvironment;
}

```

De IHostEnvironment zullen we gebruiken om een dynamisch en correct pad op te bouwen naar de locatie waar we onze file willen opslaan.

In het volgende stukje code in de AddOrUpdateImageAsync methode bouwen we een unieke filename op, na een controle te hebben uitgevoerd op de filename parameter. Als die leeg is gaat het om een nieuwe file, anders is het een update van een file.

```

if (fileName == "")
{
    fileName = $"{Guid.NewGuid()}{Path.GetExtension(image.FileName)}";
}
var pathOnDisk = Path.Combine(_hostEnvironment.ContentRootPath, "wwwroot",
    "img", "food");
if (!Directory.Exists(pathOnDisk))
{
    Directory.CreateDirectory(pathOnDisk);
}

```

Vervolgens bouwen het pad naar de directory op met behulp van de Path.Combine methode en de ContentRootPath property van de IHostEnvironment. Die bevat het correcte pad naar de wwwroot folder. De segmenten "img" en "food" worden als submappen van de wwwroot folder aangemaakt. Tot slot controleren we of de opgebouwde directory wel degelijk bestaat.

De volgende code zal dan het complete pad naar het nieuwe bestand opbouwen en door middel van de CopyToAsync methode van de IFormFile interface het opgeladen bestand naar de nieuwe locatie kopiëren. Let op het gebruik van de FileStream klasse die een stream zal openen naar het net aangemaakt pad.

```
var completePathWithFilename = Path.Combine(pathOnDisk, fileName);
using (FileStream fileStream = new(completePathWithFilename, FileMode.Create))
{
    try
    {
        await image.CopyToAsync(fileStream);
        return new ResultModel<string>
        {
            Data = fileName
        };
    }
    catch (FileNotFoundException exception)
    {
        return new ResultModel<string>
        {
            Errors = new List<string> { exception.Message }
        };
    }
}
```

Merk op dat we ook in deze serviceklasse een ResultModel object terugsturen die, wanneer de upload gelukt is, de naam van het bestand zal terugsturen in de data property.

Nu de fileservice klasse af is, gaan we een extra methode toevoegen aan de ProductsController. Om de data op te vangen die de client zal doorsturen maken we eerst een passen requestDto aan. Maak een klasse aan in de Dtos => Products folder van het Api project met als naam ProductRequestWithImageDto en plaats er onderstaande code in:

```
public class ProductRequestWithImageDto : ProductRequestDto
{
    public IFormFile Image { get; set; }
    [FileExtensions(Extensions = ".jpg,.png")]
    public string Filename => Image?.FileName;
}
```

Merk op dat we overerven van de klasse ProductRequestDto en dat we enkel een IFormFile property bijvoegen die onze image zal bijhouden. Merk ook op dat we een data annotation plaatsen bij een extra string property FileName om enkel de extensies .jpg en .png toe te laten.

Vooraleer we de ImageService kunnen gebruiken dienen we deze ook te registreren in de Program.cs:

```
builder.Services.AddScoped<IProductService, ProductService>();
builder.Services.AddScoped<ICategoryService, CategoryService>();
builder.Services.AddScoped<IImageService, ImageService>();
```

Vergeet ook niet dat we de ImageService moeten injecteren in onze ProductsController:

```
private readonly IProductService _productService;  
private readonly ICategoryService _categoryService;  
private readonly IImageService _imageService;
```

```
public ProductsController(  
    IProductService productService,  
    ICategoryService categoryService,  
    IImageService imageService)  
{  
    _productService = productService;  
    _categoryService = categoryService;  
    _imageService = imageService;  
}
```

...

Plaats vervolgens onderstaande methode in de ProductsController. Deze code is nagenoeg gelijk aan de Add methode, enkel de code in het geel werd toegevoegd om een eventuele file upload af te handelen.

```
[HttpPost("WithImage")]  
public async Task<ActionResult> AddWithImage([FromForm]ProductRequestWithImageDto productRequestWithImageDto)  
{  
    if (!await _categoryService.DoesCategoryIdExistsAsync  
        (productRequestWithImageDto.CategoryId))  
    {  
        return BadRequest($"Cannot add new product because category with id {productRequestWithImageDto.CategoryId} does not exists");  
    }  
    var product = new Product  
    {  
        CategoryId = productRequestWithImageDto.CategoryId,  
        Name = productRequestWithImageDto.Name,  
        Image = "default.jpg",  
    };  
  
    if(productRequestWithImageDto.Image != null)  
    {  
        var result = await  
            _imageService.AddOrUpdateImageAsync(productRequestWithImageDto.Image);  
        if(result.Success)  
        {  
            product.Image = result.Data;  
        }  
    }  
    var resultProduct = await _productService.AddAsync(product);  
    if (resultProduct.Success)  
    {  
        var resultCategory = await _categoryService.GetByIdAsync(product.CategoryId);  
        if (resultCategory.Success)  
        {  
            var dto = new ProductResponseDto  
            {  
                Id = product.Id,  
                Name = product.Name,  
                Category = new CategoryResponseDto  
                {  

```



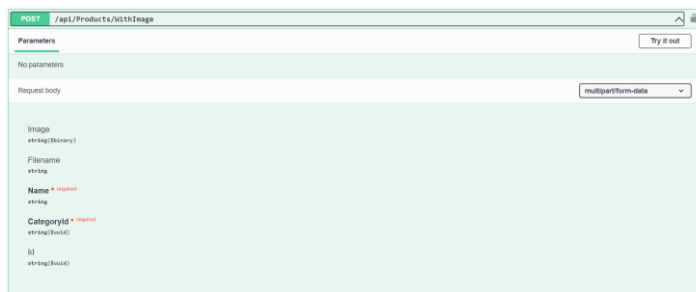
```

        Id = resultCategory.Data.Id,
        Name = resultCategory.Data.Name
    },
    Image = $"{Request.Scheme}://{Request.Host}/img/food/{product.Image}"
};
return CreatedAtAction(nameof(Get), new { id = product.Id }, dto);
}
return BadRequest(resultCategory.Errors);
}
return BadRequest(resultProduct.Errors);
}
}

```

UITTESTEN MET SWAGGER

We testen onze code even uit met Swagger. Start de applicatie en klik op de endpoint AddWithImage.



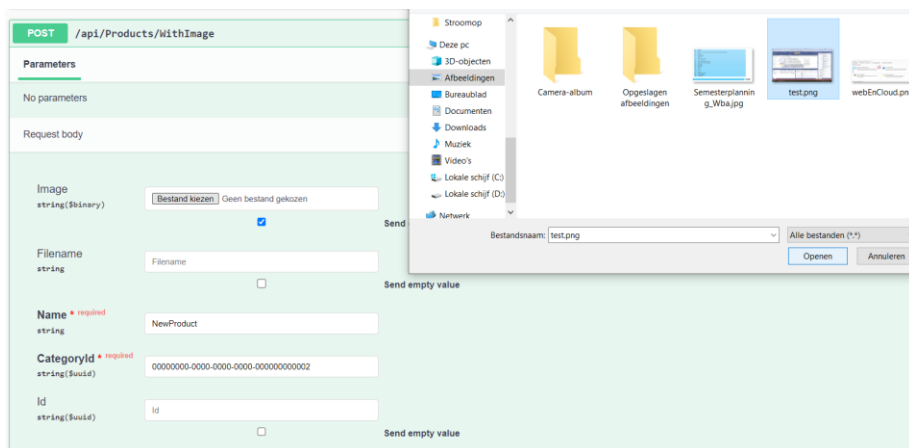
We merken al een verschil op in de interface van swagger. Dit komt door de [FromForm] annotation in de AddWithImage methode die aangeeft dat hier geen JSON verwacht wordt maar wel Form data.

```

public async Task<ActionResult> AddWithImage([FromForm] ProductRequestWithImageDto
productRequestWithImageDto)

```

Wanneer we nu op Try it out klikken zien merken we dat we een bestand kunnen toevoegen. Test eens uit en maak een nieuw product aan met een bestand, als dit goed gaat dan zal je file terug te vinden zijn in de wwwroot/img/food folder.



De velden Filename en Id laten we leeg. Wanneer je op Execute drukt zal je merken dat de api onderstaand antwoord terugstuurt

Code

Details

201
Unauthenticated

Response body

```
{
  "name": "newProduct",
  "category": {
    "name": "Pasta",
    "id": "00000000-0000-0000-0000-000000000002"
  },
  "image": "https://localhost:7011/img/food/f6618745-c89c-43ea-bb69-23c615ac2a3.png",
  "id": "8a348a62-2caa-4ef3-1501-08dc1361fc0d"
}
```

Download

Response headers

```
access-control-allow-origin: *
content-type: application/json; charset=utf-8
date: Fri, 12 Jan 2024 11:31:13 GMT
location: https://localhost:7011/api/Products/8a348a62-2caa-4ef3-1501-08dc1361fc0d
server: Kestrel
```

Responses

17 CORS (CROSS ORIGIN)

Wanneer we onze API willen testen met bijvoorbeeld de JavaScript fetch methode:

```
fetch("https://localhost:5001/api/products")
  .then(response => response.json())
  .then(data => console.log(data));
```

Zien we in onze browser dat we een error verkrijgen in onze console:



Cross-Origin-aanvraag geblokkeerd: de Same Origin Policy staat het lezen van de externe bron op <https://localhost:5001/api/products> niet toe.

(Reden: CORS-header 'Access-Control-Allow-Origin' ontbreekt).

De Web API Applicatie aanvaardt geen aanvragen van een andere Origin (domein/poortnummer).

We gaan onze Web API nu zo gaan configureren dat er wel Cross Origin aanvragen mogelijk zijn.

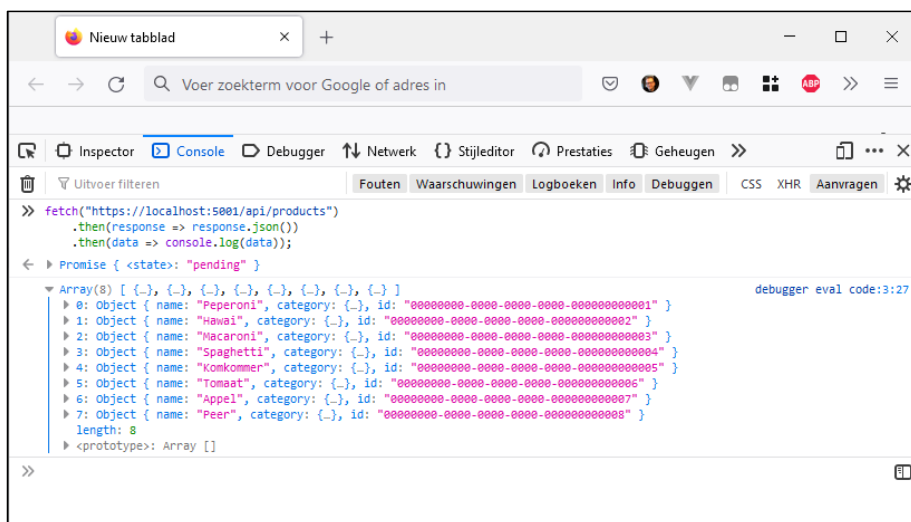
Voeg Cors toe aan de Program klasse van je API:

```
builder.Services.AddCors();
```

```
// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();

    app.UseCors(builder => builder.AllowAnyOrigin()
        .AllowAnyHeader()
        .AllowAnyMethod());
}
```

Herstart je Web API server en we zien dat we nu wel de data verkrijgen in de Console van onze browser:



howest.be